

БЕЛОРУССКИЙ НАЦИОНАЛЬНЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
РОБОТОТЕХНИКИ
КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ИНФОРМАЦИОННЫХ
СИСТЕМ И ТЕХНОЛОГИЙ»

КУРСОВАЯ РАБОТА

по дисциплине «Базы данных»

Тема: «Компьютерная фирма – Информационная система учета закупки
комплектующих»

Студент
группы 10701322

_____ Сахаревич И.Н.

Руководитель

_____ Куприянов А.Б.

Минск 2025

Белорусский национальный технический университет
Кафедра программного обеспечения информационных систем и технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

по дисциплине «Базы данных»

Тема: «Компьютерная фирма – Информационная система учета закупки комплектующих»

Исполнитель: _____ **Сахаревич И.Н.**(фамилия, инициалы)

Студент 3 курса 10701322 группы

Руководитель: _____ (фамилия, инициалы)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ.....	6
1.1 ПОСТРОЕНИЕ КОНЦЕПТУАЛЬНОЙ МОДЕЛИ	10
1.2 ПОСТРОЕНИЕ ЛОГИЧЕСКОЙ МОДЕЛИ	12
1.3 ПОСТРОЕНИЕ ФИЗИЧЕСКОЙ МОДЕЛИ	14
2 РАЗРАБОТКА ПРИЛОЖЕНИЯ ПО РАБОТЕ С БАЗОЙ ДАННЫХ....	15
2.1 НАЗНАЧЕНИЕ	15
2.2 СТРУКТУРА ПРИЛОЖЕНИЯ	16
2.3 ОПИСАНИЕ ПРИЛОЖЕНИЯ	20
2.4 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ	21
2.5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	22
3 ЗАКЛЮЧЕНИЕ	29
4 ЛИТЕРАТУРА	30
5 ПРИЛОЖЕНИЕ.....	31

ВВЕДЕНИЕ

Компьютерная фирма, занимающаяся поставкой и сборкой компьютерной техники, ежедневно сталкивается с необходимостью эффективного управления закупками комплектующих. Для обеспечения стабильной работы, своевременного пополнения складских запасов и точного учета всех операций, необходима надежная и гибкая информационная система учета закупки комплектующих.

Такая система представляет собой программный комплекс, предназначенный для автоматизации процессов взаимодействия с поставщиками, учета поступающих комплектующих, формирования и анализа данных о закупках. Она обеспечивает прозрачность закупочных операций, ускоряет обработку информации и минимизирует риски, связанные с человеческим фактором. Использование информационной системы позволяет значительно снизить затраты времени на рутинные операции, повысить точность учета и улучшить контроль над финансовыми и материальными потоками.

Информационная система учета закупок предоставляет сотрудникам доступ к актуальной информации о текущих и завершённых закупках, поставщиках, доступных категориях товаров и остатках комплектующих. Благодаря интеграции с внутренними сервисами компании, система позволяет отслеживать историю закупок, анализировать эффективность взаимодействия с поставщиками и прогнозировать потребности в тех или иных позициях.

Система позволяет компьютерной фирме вести учет всех закупаемых комплектующих, контролировать условия и объемы поставок, а также обеспечивать сотрудников актуальной информацией о ценах, поставщиках и динамике закупок. С её помощью можно оформлять и регистрировать новые закупки, вести справочники поставщиков и комплектующих, управлять условиями поставок, а также сравнивать текущие цены с предыдущими на основе данных о прошлых закупках. Кроме того, система формирует аналитические отчеты, поддерживающие принятие управленческих решений и способствующие повышению эффективности закупочной деятельности.

Кроме того, система может быть расширена модулями складского учета, бухгалтерского анализа или логистики, что позволяет организовать полноценную цифровую экосистему для эффективного функционирования компьютерной фирмы. Внедрение такой системы способствует оптимизации внутренних процессов, снижению затрат и повышению качества

обслуживания клиентов. Она не только упрощает работу сотрудников, но и обеспечивает стратегическое преимущество на рынке за счёт высокой скорости и точности обработки данных.

Цель курсовой работы – разработать информационную систему учета закупки комплектующих.

Задачи:

- спроектировать базу данных;
- построить концептуальную, логическую и физическую модели базы данных;
- разработать приложение для работы с базой данных;
- разработать руководство пользователя.

1 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

Разработка информационной системы учета закупки комплектующих начинается с тщательного проектирования базы данных, которая является ее фундаментом. Такая база должна охватывать все ключевые направления закупочной деятельности и обеспечивать надежное, структурированное хранение информации. Важным требованием является высокая скорость доступа к данным о поставщиках, комплектующих, закупках и связанных с ними деталях, что особенно критично для оперативного принятия решений.

Основу базы данных составляют таблицы, каждая из которых отвечает за определённую часть бизнес-процессов — будь то данные о поставщиках, перечне комплектующих или деталях каждой отдельной закупки. При проектировании базы важно не только определить, какие данные будут храниться, но и грамотно выстроить взаимосвязи между сущностями, чтобы обеспечить целостность и логичность хранения информации. Такой подход позволяет создать гибкую и эффективную систему, способную поддерживать стабильную работу предприятия и облегчать управление закупочной деятельностью. Для подробного объяснения на рисунке 1.1 показана диаграмма базы данных.

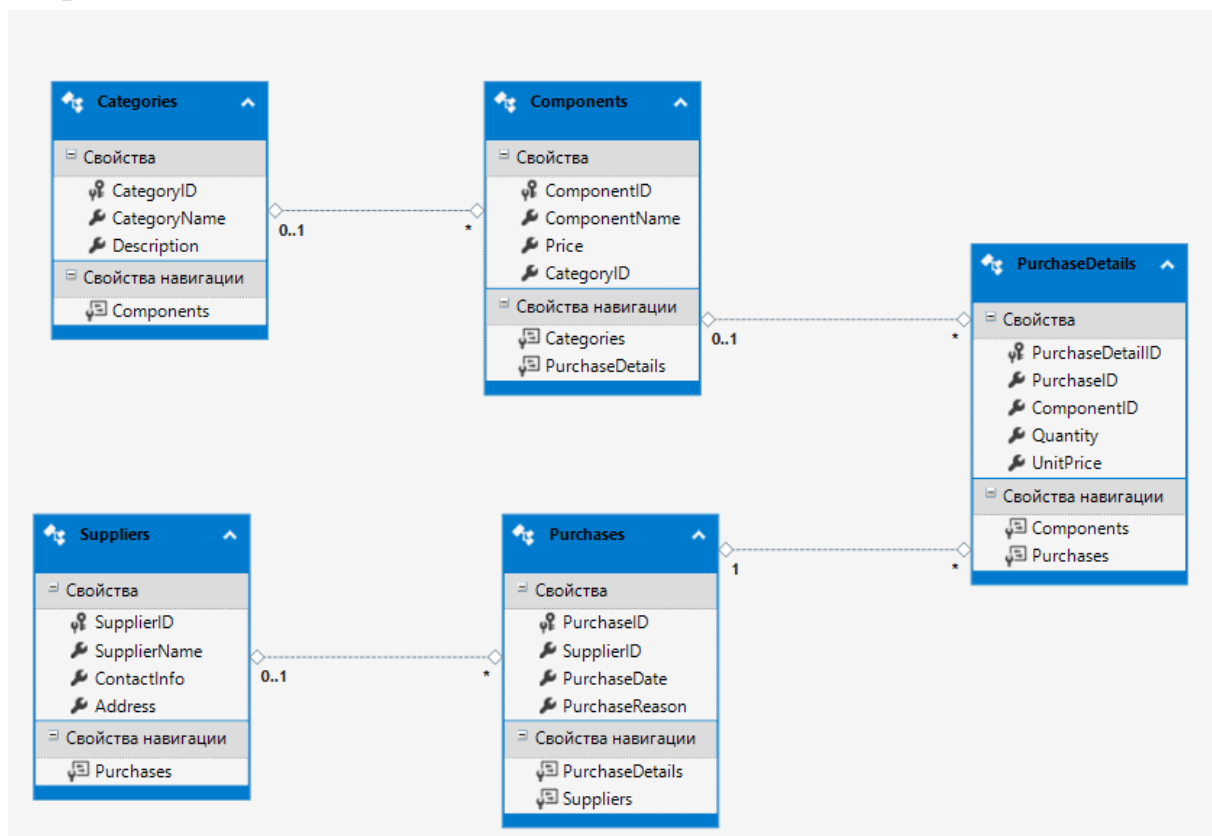


Рисунок 1.1 – Диаграмма базы данных

В базе данных, разработанной для информационной системы учета закупки комплектующих, предусмотрены логические связи между ключевыми таблицами, отражающими структуру и бизнес-процессы компьютерной фирмы. Таблица Purchases, отвечающая за регистрацию закупок, связана с таблицей Suppliers, в которой хранится информация о поставщиках, и таблицей PurchaseDetails, содержащей детализацию каждой закупки. Таблица PurchaseDetails также связана с таблицей Components, в которой представлены все комплектующие, доступные для закупки. Комплектующие, в свою очередь, классифицируются по категориям с помощью связи с таблицей Categories. Кроме таблиц, база данных содержит 4 представления, 5 хранимых процедур, 5 функции пользователя и 3 триггера.

Функции пользователя

Рассмотрим созданные функций пользователя:

GetCheapestComponent – Функция не принимает параметров и возвращает идентификатор самого дешёвого комплектующего из таблицы Components. Применяется для быстрого определения самой бюджетной позиции.

GetLastPurchaseDate – Функция принимает идентификатор поставщика и возвращает дату последней закупки у него. Используется для анализа активности взаимодействия с поставщиком.

GetPurchaseTotalCost – Данная функция принимает идентификатор закупки и возвращает её общую стоимость, рассчитывая сумму по всем позициям, указанным в деталях закупки.

GetSupplierPurchaseCount – Функция принимает идентификатор поставщика и возвращает количество закупок, совершённых у него. Полезна для статистического анализа и формирования отчётов.

GetSupplierName – Функция принимает идентификатор поставщика и возвращает его наименование. Если поставщик не найден, возвращает «Неизвестный поставщик». Может применяться для отображения текстового значения в отчётах и интерфейсах.

Представления

В базе данных создано 4 представления. Рассмотрим их:

`vw_ComponentsWithCategoryAndPrice` – Данное представление объединяет информацию о комплектующих (название, цена) с информацией о категориях, к которым они принадлежат. Таким образом, оно позволяет просматривать связь между комплектующими и их категориями, а также быстро получать сведения о стоимости.

`vw_SupplierStatistics` – Это представление содержит обобщённые данные по каждому поставщику. В нем отображаются имя поставщика, количество совершённых закупок, общее количество приобретённых товаров и итоговая сумма закупок. Представление удобно использовать для анализа эффективности взаимодействия с поставщиками.

`vw_CategoryComponentCount` – Данное представление показывает количество комплектующих в каждой категории. Оно позволяет быстро оценить, насколько каждая категория заполнена, что полезно для анализа структуры ассортимента комплектующих.

`vw_RecentPurchases` – Это представление отображает последние пять совершённых закупок. Оно включает дату, причину закупки и имя поставщика. Представление удобно для мониторинга текущих операций и получения оперативной информации по последним поставкам.

Хранимые процедуры

Хранимая процедура `AddSupplier` добавляет нового поставщика в таблицу `Suppliers`. Она принимает в качестве параметров имя поставщика (`@SupplierName`), контактную информацию (`@ContactInfo`) и адрес (`@Address`). Если контактная информация или адрес не указаны, используются значения по умолчанию: "Нет данных" для контактной информации и "Не указан" для адреса.

`AddComponent` – эта процедура добавляет новое комплектующее в таблицу `Components`. Она принимает параметры: название комплектующего (`@ComponentName`), цену комплектующего (`@Price`) и идентификатор категории (`@CategoryID`), который может быть `NULL`. Эта процедура записывает данные о комплектующем в базу данных, включая цену и категорию, если она указана.

`GetPurchaseDetailsByPurchaseID` – эта процедура возвращает подробную информацию о закупке по идентификатору закупки. Она принимает параметр

@PurchaseID, который является уникальным идентификатором закупки, и возвращает информацию о комплектующих, их количестве, цене за единицу и общей стоимости. Также в результате содержится информация о дате закупки и имени поставщика.

GetPurchaseDetailsForSuppliers – эта процедура используется для получения подробной информации о закупках для поставщиков с возможностью фильтрации по диапазону дат и идентификатору поставщика. Процедура принимает параметры @StartDate, @EndDate и @SupplierID. Она выводит информацию о закупках, включая дату, причину закупки, комплектующие, их количество, цену и категорию.

GetFilteredPurchases – эта процедура используется для получения списка закупок с возможностью фильтрации по поставщику и диапазону дат. Она принимает параметры: @SupplierID, @StartDate, @EndDate. Процедура возвращает общие данные по закупкам, включая количество и стоимость товаров в каждой закупке.

Триггеры

triggerSetUnitPrice – Данный триггер срабатывает после вставки записей в таблицу PurchaseDetails. Он автоматически устанавливает значение поля UnitPrice на основании актуальной цены комплектующего, указанной в таблице Components. Это гарантирует, что цена при покупке всегда соответствует текущей стоимости комплектующего.

triggerPreventNegativeComponentPrice – Рассматриваемый триггер срабатывает после обновления записей в таблице Components. Он проверяет, что цена комплектующего (Price) не является отрицательной. Если после обновления цена оказывается меньше нуля, триггер генерирует ошибку и отменяет транзакцию, чтобы сохранить корректность данных.

triggerCheckSupplierBeforeInsert – Текущий триггер срабатывает вместо стандартной вставки (INSTEAD OF INSERT) в таблицу Purchases. Он проверяет, существует ли поставщик с указанным SupplierID в таблице Suppliers. Если поставщик не найден, триггер генерирует ошибку. В противном случае данные вставляются в таблицу. Этот механизм предотвращает вставку записей с некорректным идентификатором поставщика.

1.1 Построение концептуальной модели

Концептуальная модель базы данных для информационной системы учёта закупок комплектующих в компьютерной фирме должна отражать основные сущности и их взаимосвязи, необходимые для эффективного управления категориями товаров, поставщиками, комплектующими и закупками.

Для графического изображения ER-диаграммы концептуальной модели базы данных можно использовать различные нотации, включая нотацию Мартина. Данная нотация является одной из наиболее распространённых и удобных для визуального отображения сущностей, их атрибутов и связей между ними. С помощью неё можно чётко представить логическую структуру данных: от категорий до деталей каждой закупки, обеспечивая наглядность и ясность при проектировании и анализе базы данных. Концептуальную модель базы данных можно увидеть на рисунке 1.2.

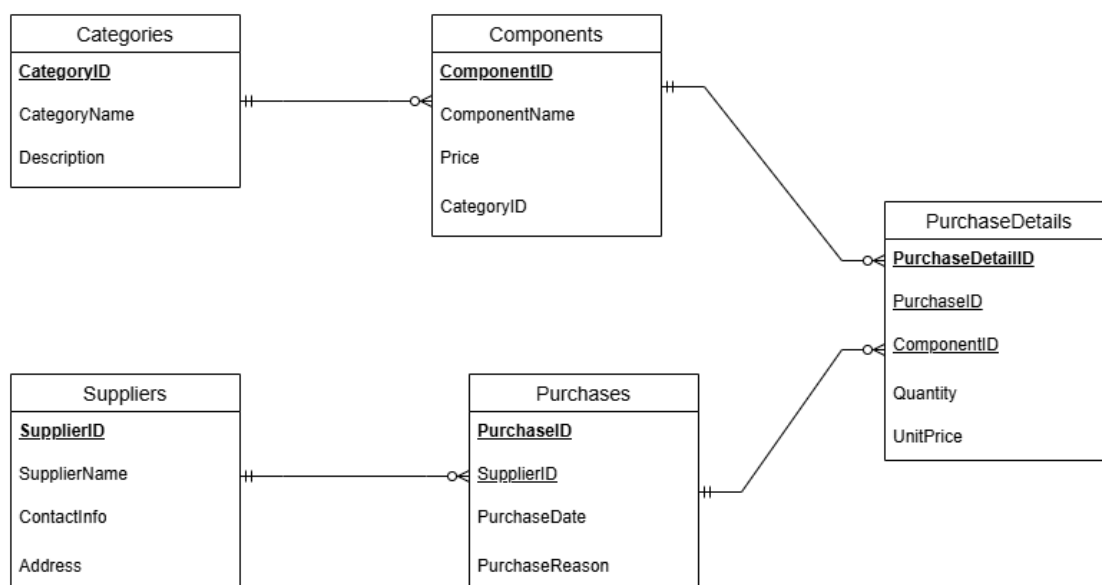


Рисунок 1.2 – Концептуальная модель базы данных

Информационная система учета вкладов в банке представляет собой комплексную систему, состоящую из нескольких ключевых сущностей. Одной из таких сущностей является клиент банка, который может открывать вклады, получать информацию о своих вкладах, а также производить различные операции с ними. Клиенты могут иметь различные типы вкладов, связанные с определенными условиями и ставками.

Другой важной сущностью является сам вклад, который хранит информацию о сумме вклада, его типе, дате открытия и других параметрах. Каждый вклад связан с конкретным клиентом и может быть открыт на определенный период времени с определенной процентной ставкой.

Также в системе присутствуют сотрудники банка, которые могут проводить операции с вкладами клиентов, вносить изменения и обеспечивать надлежащее функционирование системы учета вкладов. Каждый сотрудник имеет определенные права доступа к информации в системе в зависимости от своей должности и роли.

Концептуальная модель базы данных должна учитывать все возможные сценарии использования системы и обеспечивать эффективную и безопасную работу с данными.

Информационная система учета закупок комплектующих для компьютерной фирмы представляет собой комплексную систему, состоящую из нескольких ключевых сущностей. Одна из основных сущностей — Поставщик, который представляет организацию или компанию, поставляющую комплектующие для фирмы. Каждый поставщик может иметь несколько закупок, но каждая закупка обязана быть связана с одним поставщиком. Поставщик имеет атрибуты, такие как SupplierID (уникальный идентификатор поставщика), SupplierName (название поставщика), ContactInfo (контактная информация) и Address (адрес).

Другой важной сущностью является Комплектующее, которое представляет собой деталь или часть, закупаемую компанией. Комплектующие могут быть сгруппированы по категориям, что упрощает их учет и управление запасами. Каждое комплектующее может быть связано с одной категорией, но каждая категория может включать несколько комплектующих. Атрибуты комплектующего включают ComponentID (уникальный идентификатор комплектующего), ComponentName (название комплектующего), Price (цена комплектующего) и CategoryID (ссылка на категорию комплектующего).

Категория служит для группировки комплектующих и упрощает управление ассортиментом. Категория имеет атрибуты CategoryID (уникальный идентификатор категории), CategoryName (название категории) и Description (описание категории).

Закупка — это процесс приобретения комплектующих у поставщика. Каждая закупка может включать несколько деталей закупки, каждая из которых будет содержать информацию о комплектующих, их количестве и цене. Закупка связана с одним поставщиком. Атрибуты закупки включают PurchaseID (уникальный идентификатор закупки), SupplierID (ссылка на

поставщика), PurchaseDate (дата создания закупки) и PurchaseReason (причина закупки).

Детали закупки содержат информацию о комплектующих, приобретенных в рамках закупки. Каждая деталь закупки обязана быть связана с одной закупкой и с одним комплектующим. Атрибуты журнала закупки включают PurchaseDetailID (уникальный идентификатор детали закупки), PurchaseID (ссылка на закупку), ComponentID (ссылка на комплектующее), Quantity (количество комплектующего в закупке) и UnitPrice (цена за единицу комплектующего).

Таким образом, в системе реализованы связи между сущностями: каждый поставщик может иметь несколько закупок, но каждая закупка обязана быть связана с одним поставщиком; каждая закупка может содержать несколько деталей закупки, каждая из которых обязательно связана с одной закупкой; одно комплектующее может быть частью многих деталей закупки, но каждая деталь закупки содержит только одно комплектующее; и каждое комплектующее принадлежит одной категории, но каждая категория может включать несколько комплектующих. Эта структура обеспечивает эффективное управление закупками комплектующих и их учетом в компании.

1.2 Построение логической модели

Логическая модель базы данных — это модель, которая описывает структуру базы данных. В отличие от концептуальной модели, логическая модель уже содержит более детальную информацию о структуре данных и отношениях между ними, также она отражает структуру базы данных, готовую к созданию в СУБД.

Логическая модель базы данных включает в себя определение всех сущностей, их атрибутов, типов данных, а также связей между ними.

Поле	Тип данных	Ограничения
CategoryID	INT	PRIMARY KEY, IDENTITY(1, 1),
CategoryName	NVARCHAR(100)	NOT NULL, UNIQUE
Description	NVARCHAR(256)	

Таблица 1.1 Структура таблицы Categories

Поле	Тип данных	Ограничения
ComponentID	INT	PRIMARY KEY, IDENTITY(1, 1)
ComponentName	NVARCHAR(100)	NOT NULL, UNIQUE
Price	DECIMAL(10, 2)	NOT NULL, CHECK (Price >= 0)

CategoryID	INT	FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID) ON DELETE SET NULL ON UPDATE CASCADE
-------------------	-----	--

Таблица 1.2 Структура таблицы Components

Поле	Тип данных	Ограничения
SupplierID	INT	PRIMARY KEY, IDENTITY(1, 1)
SupplierName	NVARCHAR(100)	NOT NULL, UNIQUE
ContactInfo	NVARCHAR(100)	NOT NULL, DEFAULT 'Нет данных'
Address	NVARCHAR(255)	NOT NULL, DEFAULT 'Не указан'

Таблица 1.3 Структура таблицы Suppliers

Поле	Тип данных	Ограничения
PurchaseID	INT	PRIMARY KEY, IDENTITY(1, 1)
SupplierID	INT	FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID) ON DELETE SET NULL ON UPDATE CASCADE
PurchaseDate	Date	DEFAULT GETDATE()
PurchaseReason	NVARCHAR(255)	NOT NULL

Таблица 1.4 Структура таблицы Purchases

Поле	Тип данных	Ограничения
PurchaseDetailID	INT	PRIMARY KEY, IDENTITY(1, 1)
PurchaseID	INT	FOREIGN KEY (PurchaseID) REFERENCES Purchases(PurchaseID) ON DELETE CASCADE ON UPDATE CASCADE, NOT NULL
ComponentID	INT	FOREIGN KEY (ComponentID) REFERENCES Components(ComponentID) ON DELETE SET NULL ON UPDATE CASCADE
Quantity	INT	CHECK (Quantity > 0), NOT NULL

UnitPrice	DECIMAL(10, 2)	CHECK (UnitPrice >= 0), NOT NULL, DEFAULT 0
------------------	----------------	---

Таблица 1.5 Структура таблицы PurchaseDetails

1.3 Построение физической модели

Физическая модель базы данных представляет собой конкретную реализацию логической модели в виде физических объектов базы данных, таких как таблицы, столбцы, индексы и ограничения. Она определяет, как данные будут храниться на диске и как они будут доступны для чтения и записи программным обеспечением.

Физическая модель базы данных вкладов:

1. Имя базы данных: ComputerCompanyDB
2. Имя журнала транзакций: ComputerCompanyDB_log
3. Путь к файлу базы данных: C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\ComputerCompanyDB.mdf
4. Путь к журналу транзакций: C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\ComputerCompanyDB_log.ldf
5. Начальный размер файла базы данных и журнала транзакций: 8 мб
6. Шаг приращения файла базы данных и журнала транзакций: 64 мб

2 РАЗРАБОТКА ПРИЛОЖЕНИЯ ПО РАБОТЕ С БАЗОЙ ДАННЫХ

При разработке информационной системы учета закупок комплектующих для компьютерной фирмы необходимо определить требования к функциональности системы и спроектировать соответствующую архитектуру, которая будет основывать работу системы на структуре базы данных. Система должна обеспечивать полный учет и управление закупками комплектующих, а также поддерживать управление категориями комплектующих и поставщиками. Важно, чтобы система могла отслеживать все детали закупок и корректно работать с информацией о каждой закупке.

Кроме того, система должна позволять создавать и редактировать записи о поставщиках и комплектующих, а также поддерживать возможность вывода отчетов по закупкам. Например, пользователь должен иметь возможность просматривать все закупки, связанные с конкретным поставщиком, а также видеть подробности о каждом комплектующем в контексте конкретной закупки.

Система должна включать функционал для интеграции с другими бизнес-процессами компании, например, для учета поставок комплектующих или управления финансовыми операциями, связанными с закупками. Необходимо предусмотреть механизмы защиты данных и обработки ошибок, чтобы обеспечить надежность работы системы. Важным аспектом является возможность тестирования системы для выявления и устранения ошибок. Для этого могут быть использованы различные технологии тестирования, включая модульное и функциональное тестирование. Также необходимо проводить интеграционное тестирование с другими системами фирмы, например, с системой управления поставщиками, для обеспечения корректной работы всех компонентов системы.

2.1 Назначение

Целью информационной системы учета закупок комплектующих для компьютерной фирмы является автоматизация процессов, связанных с закупкой и управлением комплектующими. Система должна обеспечивать возможность отслеживания данных о комплектующих, их стоимости, категориях и поставщиках, а также подробное фиксирование информации о закупках и их деталях. Она позволит сотрудникам компании эффективно управлять данными о поставках, а также контролировать количество и цену комплектующих. В результате внедрения системы будет повышена эффективность работы сотрудников, уменьшено количество ошибок при учете

и ускорен процесс обработки данных, что в свою очередь улучшит общее управление закупками и взаимодействие с поставщиками.

2.2 Структура приложения

Приложение состоит из главной формы, форм для работы с таблицами, формы для вывода отчета, перемещение по которым осуществляется с помощью кнопок из связанных форм или главной формы. Для вывода информации в таблицах используется DataGridView, отчет формируется с помощью ReportViewer.

Главная форма – MainForm

Данная форма содержит кнопки для перехода на другие формы, в которых можно будет работать с данными в таблицах, также имеется возможность выводить отчеты. Данная форму можно увидеть на рисунке 2.1.

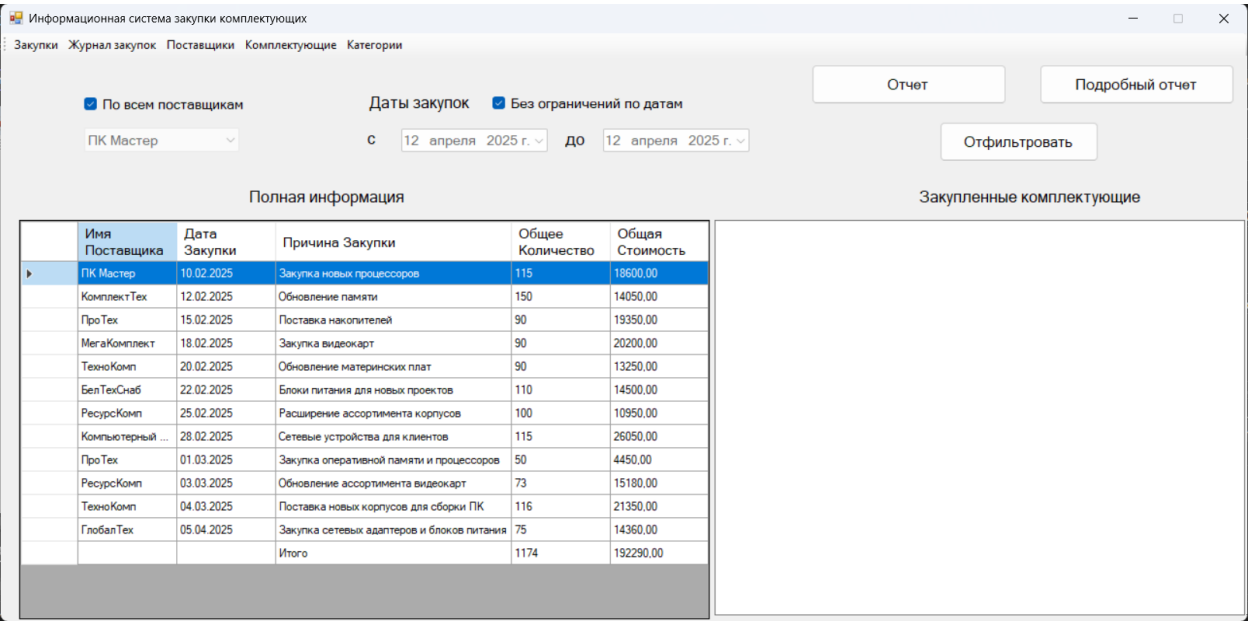
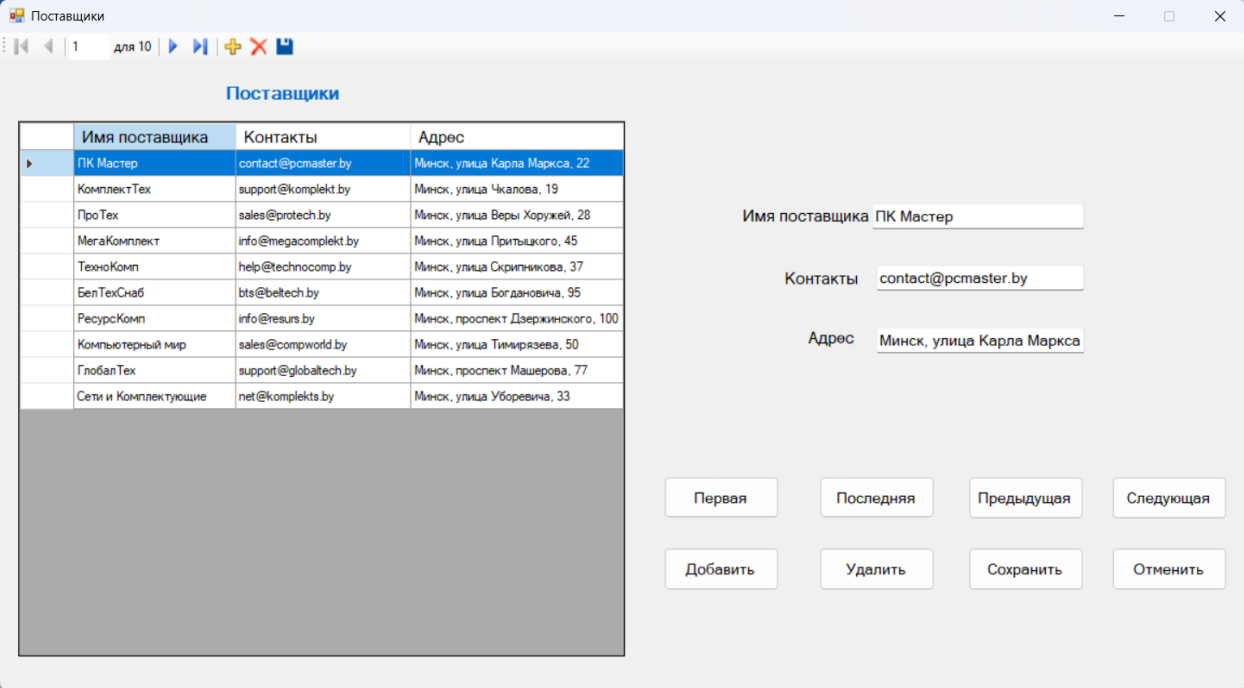


Рисунок 2.1 – Главная форма

Форма для работы с таблицей поставщики – SuppliersForm

Данная форма предназначена для управления данными таблицы поставщиков. С её помощью можно добавлять, изменять и удалять записи, а также сохранять изменения в базе данных. Структура данной формы показана на рисунке 2.2.



Поставщики

Имя поставщика	Контакты	Адрес
ПК Мастер	contact@pcmaster.by	Минск, улица Карла Маркса, 22
КомплектТех	support@komplekt.by	Минск, улица Чкалова, 19
ПроТех	sales@protech.by	Минск, улица Веры Хоружей, 28
МегаКомплект	info@megacomplekt.by	Минск, улица Притыцкого, 45
ТехноКомп	help@technocomp.by	Минск, улица Скрипникова, 37
БелТехСнаб	bts@beltech.by	Минск, улица Богдановича, 95
РесурсКомп	info@resurs.by	Минск, проспект Дзержинского, 100
Компьютерный мир	sales@compworld.by	Минск, улица Тимирязева, 50
ГлобалТех	support@globaltech.by	Минск, проспект Машерова, 77
Сети и Комплектующие	net@komplekts.by	Минск, улица Уборевича, 33

Имя поставщика: ПК Мастер

Контакты: contact@pcmaster.by

Адрес: Минск, улица Карла Маркса

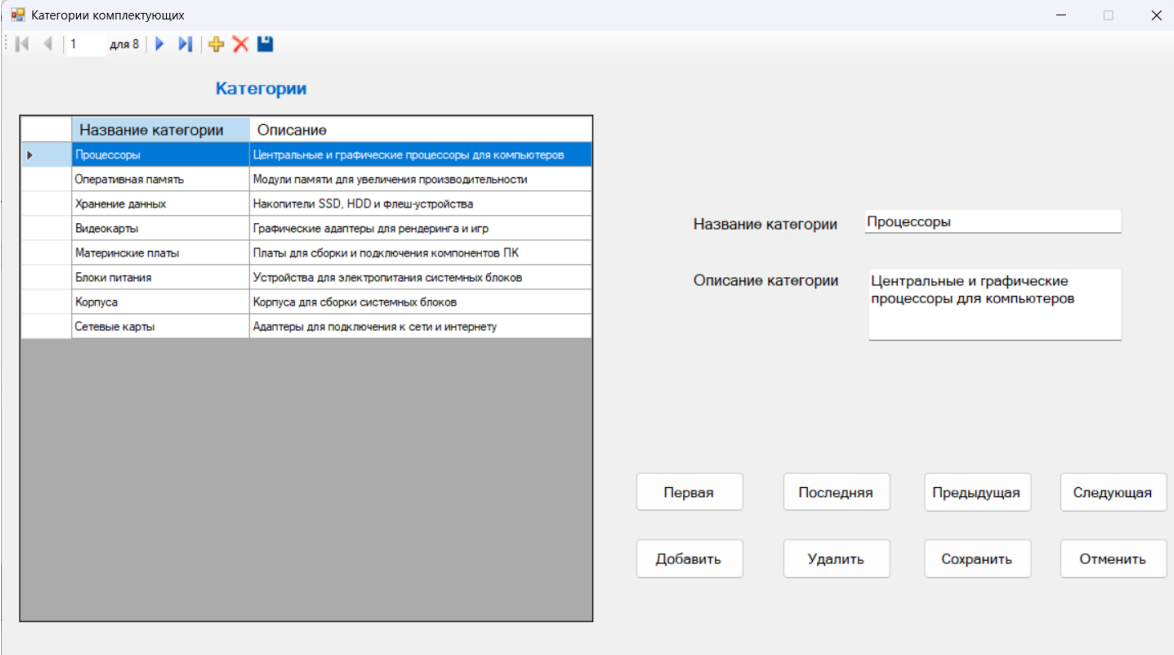
Первая Последняя Предыдущая Следующая

Добавить Удалить Сохранить Отменить

Рисунок 2.2 – Форма поставщиков

Форма для работы с таблицей категории – CategoriesForm

Форма используется для работы с таблицей категорий. Она предоставляет возможность добавления новых записей, редактирования и удаления существующих, а также сохранения внесённых изменений в таблице. Данная форма показана на рисунке 2.3.



Категории

Название категории	Описание
Процессоры	Центральные и графические процессоры для компьютеров
Оперативная память	Модули памяти для увеличения производительности
Хранение данных	Накопители SSD, HDD и флеш-устройства
Видеокарты	Графические адаптеры для рендеринга и игр
Материнские платы	Платы для сборки и подключения компонентов ПК
Блоки питания	Устройства для электропитания системных блоков
Корпуса	Корпуса для сборки системных блоков
Сетевые карты	Адаптеры для подключения к сети и интернету

Название категории: Процессоры

Описание категории: Центральные и графические процессоры для компьютеров

Первая Последняя Предыдущая Следующая

Добавить Удалить Сохранить Отменить

Рисунок 2.3 – Форма категорий

Форма для работы с таблицей номера – ComponentsForm

Форма предназначена для управления таблицей комплектующих. Позволяет добавлять, редактировать и удалять записи по определенной категории, а также сохранять изменения в базе данных. Форма комплектующих показана на рисунке 2.4, где можно увидеть все важные элементы структуры.

Комплектующие

Категория: Процессоры

Категории

Название комплектующего	Цена
Intel Core i5-12600K	300.00
AMD Ryzen 5 5600X	280.00
AMD Ryzen 9 5900X	450.00

Название комплектующего: Intel Core i5-12600K

Цена: 300.00

Первая
Последняя
Предыдущая
Следующая

Добавить
Удалить
Сохранить
Отменить

Рисунок 2.4 – Форма комплектующих

Форма для работы с таблицей закупок – PurchasesForm

Данная форма предназначена для управления данными таблицы закупок. Она предназначена для управления данными таблицы закупок, позволяя добавлять новые записи, изменять и удалять существующие, а также сохранять внесённые изменения в базе данных. Структура данной формы показана на рисунке 2.5.

Закупки

1 для 12

Закупки

Имя поставщика	Дата покупки	Причина закупки
ПК Мастер	10.02.2025	Закупка новых процессоров
КомплектТех	12.02.2025	Обновление памяти
ПроТех	15.02.2025	Поставка накопителей
МегаКомплект	18.02.2025	Закупка видеокарт
ТехноКомп	20.02.2025	Обновление материнских плат
БелТехСнаб	22.02.2025	Блоки питания для новых проектов
РесурсКомп	25.02.2025	Расширение ассортимента корпусов
Компьютерный мир	28.02.2025	Сетевые устройства для клиентов
ПроТех	01.03.2025	Закупка оперативной памяти и процессоров
РесурсКомп	03.03.2025	Обновление ассортимента видеокарт
ТехноКомп	04.03.2025	Поставка новых корпусов для сборки ПК
ГлобалТех	05.04.2025	Закупка сетевых адаптеров и блоков питания

Номер покупки

1

Имя поставщика

ПК Мастер

Список поставщиков

Дата покупки

10 февраля 2025 г.

Причина оформления закупки

Закупка новых процессоров

Первая

Последняя

Предыдущая

Следующая

Добавить

Удалить

Сохранить

Отменить

Рисунок 2.5 – Форма закупок

Форма для работы с журналом закупок – PurchaseDetailsForm

Эта форма предназначена для эффективного управления записями детали закупок. Она обеспечивает возможность добавления, редактирования и удаления записей по определенному номеру закупки, что превращает эту форму в полноценный журнал закупки и гарантирует актуальность данных в таблице. Также в этой форме имеется возможность вывести чек, в котором показывается все комплектующие, которые мы закупили, их количество и сумма, также дата создания покупки, поставщика, который принял за этот заказ, и сам номер закупки. Изображение формы можно найти на рисунке 2.6.

Журнал закупок

1 для 4

Журнал закупок

Номер закупки: 1

Закупки

Название комплектующего	Количество	Цена за единицу
Intel Core i5-12600K	25	300,00
AMD Ryzen 5 5600X	20	280,00
Corsair Vengeance 8GB DDR4	30	50,00
Kingston Fury Beast 16GB DDR5	40	100,00

Название комплектующего: Intel Core i5-12600K

Список компонентов

Количество: 25

Стоимость за эту деталь: 300,00

Первая Последняя Предыдущая Следующая

Добавить Удалить Сохранить Отменить

Выдать чек

Рисунок 2.6 – Форма журнала закупок

2.3 Описание приложения

Приложение, разработанное для учета закупок комплектующих в компьютерной фирме, предназначено для взаимодействия пользователей с базой данных, контроля закупок, управления номенклатурой и анализа данных о поставках.

С помощью приложения пользователь может просматривать информацию о поставщиках, комплектующих, закупках и их деталях, а также добавлять, изменять и удалять соответствующие данные. Реализована возможность анализа истории закупок, отслеживания количества приобретённого оборудования и его стоимости.

Программа поддерживает формирование табличных отчетов, которые можно использовать для печати и анализа затрат за определенный период времени, а также для оценки активности поставщиков и планирования дальнейших закупок.

Таким образом, приложение для учета закупок комплектующих представляет собой полноценное средство автоматизации закупочной деятельности, обеспечивающее прозрачность, удобство и эффективность процессов снабжения.

2.4 Тестирование приложения

Тестирование приложения для информационной системы учёта закупок комплектующих в компьютерной фирме является важным этапом разработки, позволяющим выявить и устранить ошибки в логике работы программы, взаимодействии с базой данных и пользовательском интерфейсе.

Особое внимание при тестировании уделяется корректности добавления, изменения и удаления данных о поставщиках, закупках, комплектующих и категориях. Тестирование помогает убедиться в целостности данных, правильной реализации связей между таблицами и корректной обработке пользовательского ввода.

В приложении предусмотрены обработчики событий, сопровождающие нажатия на кнопки добавления записей. При некорректном или неполном заполнении обязательных полей пользователю отображается окно с сообщением об ошибке (MessageBox), что позволяет избежать некорректного ввода и сохранить целостность данных. Данное окно с сообщением об ошибке приведено в рисунке 2.7.

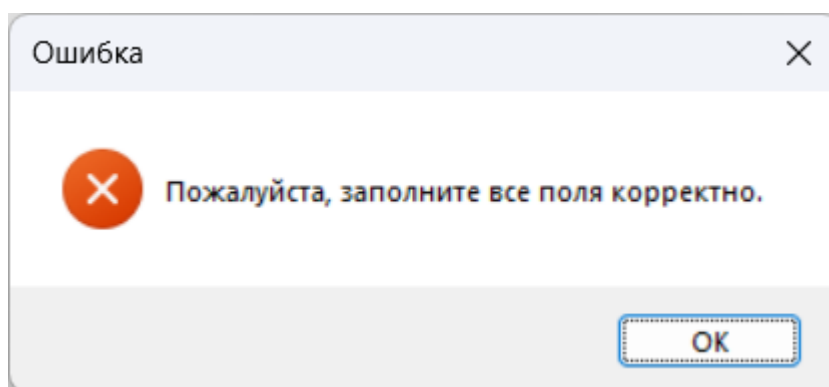


Рисунок 2.7 – Сообщение о корректности данных

Тестирование проводится на различных этапах разработки, включая модульное, интеграционное и системное тестирование. Полученные результаты документируются и анализируются для последующего улучшения как пользовательской части приложения, так и структуры базы данных.

2.5 Руководство пользователя

При запуске программы будет отображено главное окно, где происходят основные манипуляции с программой. Форма иллюстрирована на рисунке 2.8

Имя Поставщика	Дата Закупки	Причина Закупки	Общее Количество	Общая Стоимость
ПК Мастер	10.02.2025	Закупка новых процессоров	115	18600.00
КомплектТех	12.02.2025	Обновление памяти	150	14050.00
ПроТех	15.02.2025	Поставка накопителей	90	19350.00
МегаКомплект	18.02.2025	Закупка видеокарт	90	20200.00
ТехноКомп	20.02.2025	Обновление материнских плат	90	13250.00
БелТехСнаб	22.02.2025	Блоки питания для новых проектов	110	14500.00
РесурсКомп	25.02.2025	Расширение ассортимента корпусов	100	10950.00
Компьютерный ...	28.02.2025	Сетевые устройства для клиентов	115	26050.00
ПроТех	01.03.2025	Закупка оперативной памяти и процессоров	50	4450.00
РесурсКомп	03.03.2025	Обновление ассортимента видеокарт	73	15180.00
ТехноКомп	04.03.2025	Поставка новых корпусов для сборки ПК	116	21350.00
ГлобалТех	05.04.2025	Закупка сетевых адаптеров и блоков питания	75	14360.00
		Итого	1174	192290.00

Рисунок 2.8 – Форма «Главная»

Данная форма необходима формирования и вывода информации об итогах работы фирмы за выбираемый период и по выбранному поставщику. На форме имеются кнопки для перехода на связанные таблицы: закупки, журнал закупок, поставщики, категории.

Для вывода отчета за период времени и выбранному поставщику так же существуют соответствующие кнопки, но сейчас мы рассмотрим процесс добавления информации на примере формы работы с поставщиками, представленной на рисунке 2.9.

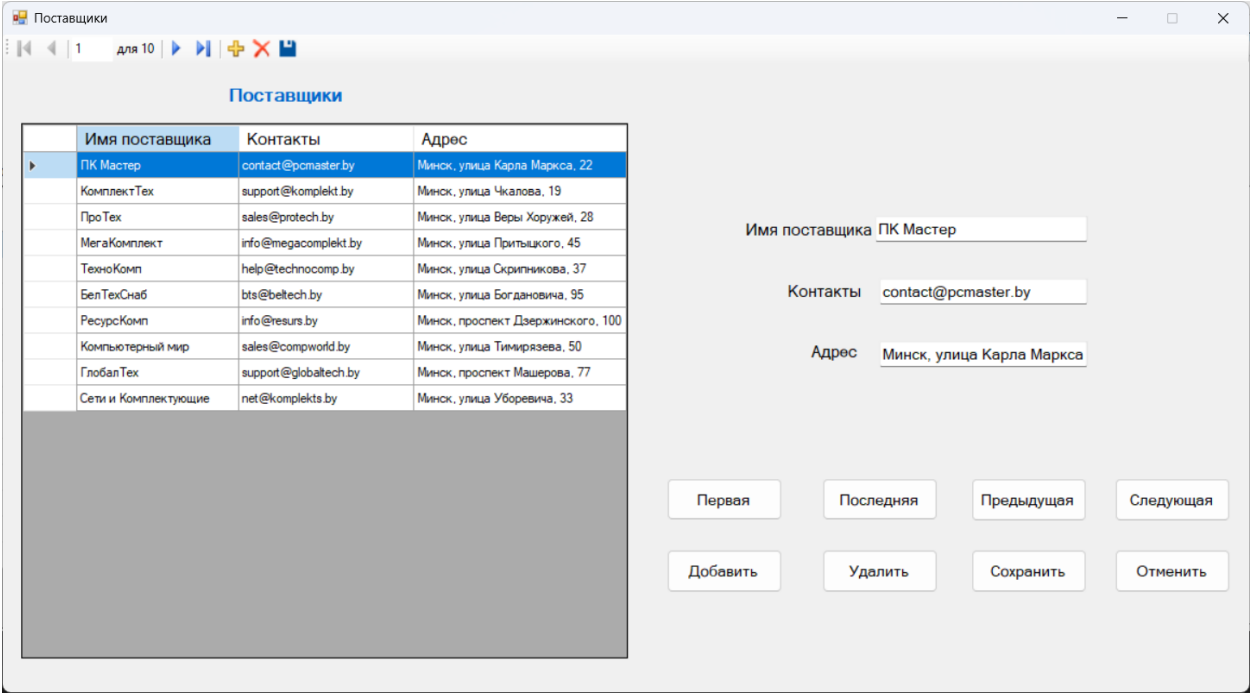


Рисунок 2.9 – Форма поставщиков

Для добавления данных нужно ввести нового поставщика. Для этого в окне нажимаем кнопку «Добавить», у нас открывается форма добавления поставщика, в которой мы указываем всю информацию о новом поставщике, после чего нажать на кнопку «Добавить». Рисунок 2.10 иллюстрирует форму создания новой записи.

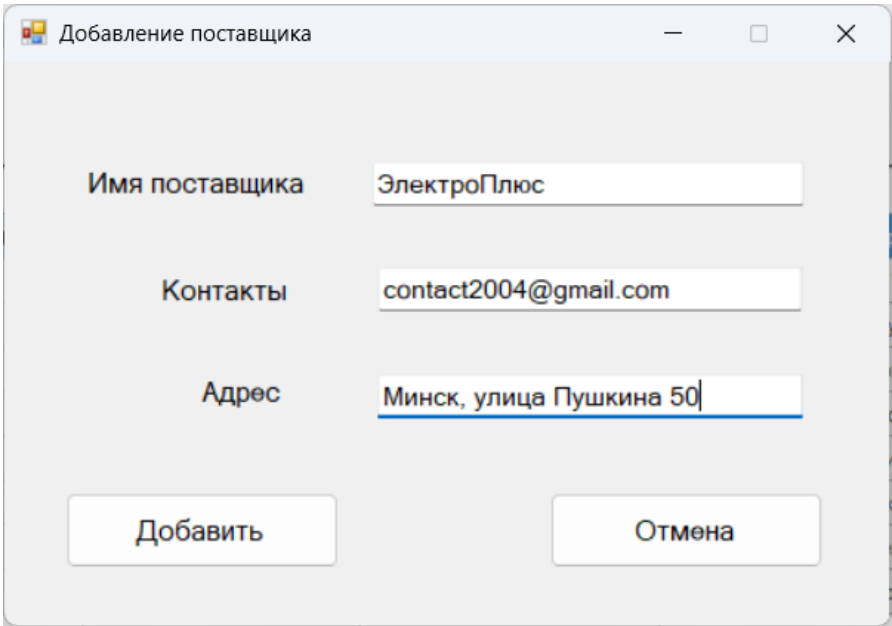


Рисунок 2.10 – Добавление нового поставщика

После добавления в рабочей форме появиться новый поставщик в нашей таблице, но не стоит забывать, что это несохраненная информация. Поэтому надо после ее добавления сохранить ее в базу данных. Форма с новой записью показана на рисунке 2.11.

Имя поставщика	Контакты	Адрес
ПК Мастер	contact@pcmaster.by	Минск, улица Карла Маркса, 22
КомплектТех	support@komplekt.by	Минск, улица Чкалова, 19
ПроТех	sales@protech.by	Минск, улица Веры Хоружей, 28
МегаКомплект	info@megacomplekt.by	Минск, улица Притыцкого, 45
ТехноКомп	help@technocomp.by	Минск, улица Скрипникова, 37
БелТехСнаб	bta@beltech.by	Минск, улица Богдановича, 95
РесурсКомп	info@resurs.by	Минск, проспект Дзержинского, 100
Компьютерный мир	sales@compworld.by	Минск, улица Тимирязева, 50
ГлобалТех	support@globaltech.by	Минск, проспект Машерова, 77
Сети и Комплектующие	net@komplekts.by	Минск, улица Уборевича, 33
ЭлектроПлюс	contact2004@gmail.com	Минск, улица Пушкина 50

Имя поставщика: ЭлектроПлюс

Контакты: contact2004@gmail.com

Адрес: Минск, улица Пушкина 50

Первая Последняя Предыдущая Следующая

Добавить Удалить Сохранить Отменить

Рисунок 2.11 – Форма поставщиков с новой записью

Теперь нам необходимо обратно перейти на главную форму, которая изображена на рисунке 2.1. Для вывода отчета нужно нажать на кнопку «Отчет», которая откроет форму отчета. Этот отчет был сформирован по выбранным параметрам. Данное окно показано на рисунке 2.12.

Окно отчета

1 из 1 100% Найти | Следующий

Отчет по закупкам комплектующих с 01 фев 2025 до 20 фев 2025
для всех поставщиков

	Поставщик	Дата закупки	Причина закупки	Количество	Стоимость
	ПК Мастер	10 февраля 2025 г.	Закупка новых процессоров	115	18600,00
	КомплектТех	12 февраля 2025 г.	Обновление памяти	150	14050,00
	ПроТех	15 февраля 2025 г.	Поставка накопителей	90	19350,00
	МegaКомплект	18 февраля 2025 г.	Закупка видеокарт	90	20200,00
	ТехноКомп	20 февраля 2025 г.	Обновление материнских плат	90	13250,00
Всего				535	85450,00

Рисунок 2.12 – Окно вывода отчета

После чего нужно нажать иконку сохранения, выбрать нужный тип отчета (PDF, Word, Excel) и после выведется диалоговое окно сохранения, показанное на рисунке 2.13.

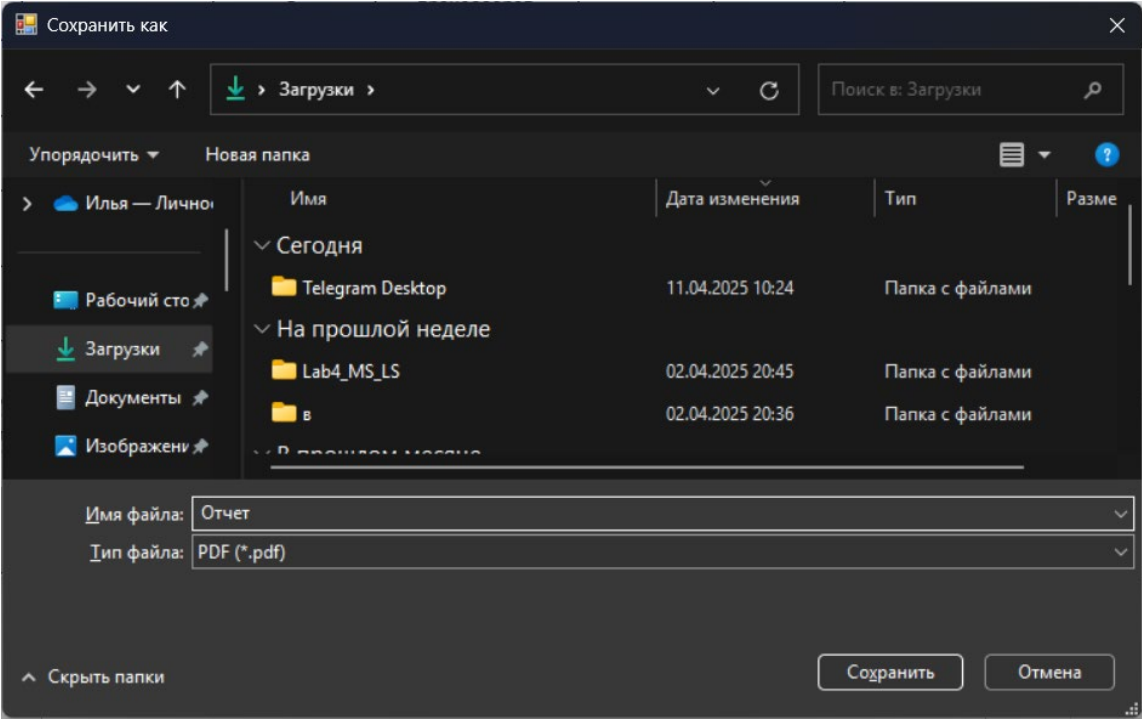


Рисунок 2.13 – Сохранение отчета

После выполнения данного действия пользователю отображается диалоговое окно (MessageBox) с вопросом: открыть ли выбранный документ, показанный на рисунке 2.14.

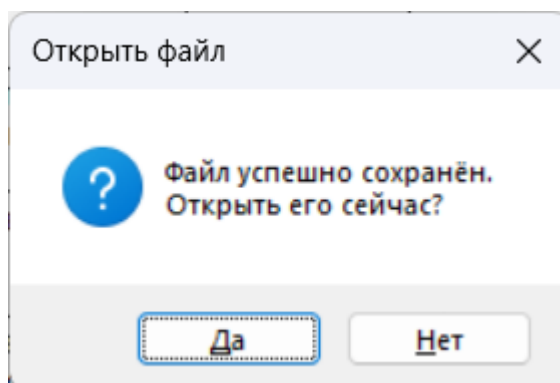


Рисунок 2.14 – MessageBox с запросом на открытие документа

После нажатия кнопки «Да» в MessageBox происходит открытие выбранного документа, который можно увидеть на рисунке 2.15.

**Отчет по закупкам комплектующих с 01 фев 2025 до 24 фев 2025
для всех поставщиков**

	Поставщик	Дата закупки	Причина закупки	Количество	Стоимость
	ПК Мастер	10 февраля 2025 г.	Закупка новых процессоров	115	18600,00
	КомплектТех	12 февраля 2025 г.	Обновление памяти	150	14050,00
	ПроТех	15 февраля 2025 г.	Поставка накопителей	90	19350,00
	МегаКомплект	18 февраля 2025 г.	Закупка видеокарт	90	20200,00
	ТехноКомп	20 февраля 2025 г.	Обновление материнских плат	90	13250,00
	БелТехСнаб	22 февраля 2025 г.	Блоки питания для новых проектов	110	14500,00
Всего				645	99950,00

Рисунок 2.15 – Отчет за выбранный промежуток времени по всем поставщикам

Также на главной форме у нас кнопка “Подробный отчет”, открывающая отчет, но уже с подробной информацией о закупках. Данный отчет открывается в таком же окне, но уже имеет совершенно другой вид. Указанный отчет можно увидеть на рисунке 2.16.

Отчет по закупкам комплектующих с 01 фев 2025 до 24 фев 2025 для поставщика ПК Мастер

Дата закупки: 10 февраля 2025 г.

Причина закупки: Закупка новых процессоров

Общее количество: 115

Общая стоимость: 18600,00

Название комплектующего	Категория	Цена за единицу	Количество	Итого
AMD Ryzen 5 5600X	Процессоры	280,00	20	5600,00
Corsair Vengeance 8GB DDR4	Оперативная память	50,00	30	1500,00
Intel Core i5-12600K	Процессоры	300,00	25	7500,00
Kingston Fury Beast 16GB DDR5	Оперативная память	100,00	40	4000,00
Всего			115	18600,00

Рисунок 2.16 – Подробный отчет за выбранный промежуток времени по определенному поставщику

Также в форме журнала покупки как было написано ранее можно вывести чек закупки при нажатии на кнопку «Вывести чек». Данный чек представлен на рисунке 2.17.

ООО "ИТТехИлья"
Магазин техники "ТехноКупол"
г. Минск, ул. Притыцкого 97
Режим работы с 9.00 до 22.00

ЧЕК ЗАКУПКИ
ПЛАТЕЖНЫЙ ДОКУМЕНТ

Дата покупки: 10 февраля 2025 г.

Название компании или ЧЛ поставившего товары: ПК Мастер

Номер чека: 1

Комплектующие	Количество	Цена за единицу	Сумма
Intel Core i5-12600K	25	300,00	7500,00
AMD Ryzen 5 5600X	20	280,00	5600,00
Corsair Vengeance 8GB DDR4	30	50,00	1500,00
Kingston Fury Beast 16GB DDR5	40	100,00	4000,00
Итого			18600,00

Было приятно иметь дело с вами!

Рисунок 2.17 – Подробный отчет за выбранный промежуток времени по определенному поставщику

3 ЗАКЛЮЧЕНИЕ

В завершение разработки можно отметить, что была создана полноценная информационная система, предназначенная для автоматизации процесса учёта закупок комплектующих в компьютерной фирме. На первоначальном этапе была проведена аналитика предметной области, на основе которой сформированы требования к структуре базы данных и функциональности приложения.

Далее было выполнено поэтапное проектирование базы данных: сформированы концептуальная, логическая и физическая модели, которые легли в основу реализации системы. Приложение разработано с использованием среды Microsoft Visual Studio, языка программирования C#, технологии ADO.NET и СУБД SQL Server, что обеспечило удобство работы с данными и стабильность функционирования системы.

В ходе реализации были внедрены функции добавления, редактирования, удаления и поиска записей, а также формирование отчётной документации. Для повышения надёжности и удобства в интерфейс были встроены механизмы обработки ошибок, в том числе визуальное информирование пользователя через диалоговые окна (MessageBox).

Проведённое тестирование показало, что система работает корректно, соответствует заявленным требованиям и способна обрабатывать реальные данные без сбоев. Она обеспечивает удобный инструмент для управления закупками, анализа информации и взаимодействия с поставщиками.

Таким образом, созданная информационная система значительно упрощает учёт и контроль за поступлением комплектующих, способствует оптимизации бизнес-процессов и может стать важным элементом в работе компьютерной фирмы.

4 ЛИТЕРАТУРА

1. Агуров, Павел. С#. Сборник рецептов / Павел Агуров. - М.: "БХВ Петербург", 2012. – 432 с.
2. Тепляков, Виктор. С# 7.0 для профессионалов / Виктор Тепляков. - М.: ООО "Издательство Эксмо", 2018. - 800 с.
3. Шилдт, Герберт. С# 6.0. Справочное руководство / Герберт Шилдт. - М.: ООО "Издательство Эксмо", 2016. - 592 с.
4. Романенко, Сергей. С# 7.0. Карманный справочник / Сергей Романенко. - СПб.: БХВ-Петербург, 2017. - 352 с.
5. Кузнецов, Александр. Программирование на С# в Visual Studio 2019 для профессионалов / Александр Кузнецов. - М.: ООО "Издательство Эксмо", 2020. - 896 с.
6. Коннелли, Джеймс. SQL. Полное руководство / Джеймс Коннелли. - М.: ООО "Издательство Эксмо", 2019. - 960 с.
7. Фаулер, Грегори. Шаблоны корпоративных приложений / Грегори Фаулер. - М.: "Издательство ДМК Пресс", 2003. - 512 с.
8. Хантер, Андрей. Windows Forms в действии. Программирование пользовательских интерфейсов в .NET / Андрей Хантер. - М.: "Издательский дом "Вильямс"", 2006. - 528 с.
9. Лиддерс, Эрик. Entity Framework 6. Программирование баз данных / Эрик Лиддерс. - М.: "ДМК Пресс", 2016. - 384 с.
10. Шило, Андрей. SQL и процедурно-ориентированные языки / Андрей Шило. - М.: Бином, 2012. - 352 с.

5 ПРИЛОЖЕНИЕ

Форма CategoriesForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class CategoriesForm : Form
    {
        private AddCategoriesForm addCategoriesForm;
        private bool promptOnClose; // Флаг для показа MessageBox
        public CategoriesForm(bool promptOnClose = false)
        {
            InitializeComponent();
            this.promptOnClose = promptOnClose;
        }

        private void SaveCategoriesData()
        {
            try
            {
                this.Validate();
                this.categoriesBindingSource.EndEdit();
                this.tableAdapterManager.UpdateAll(this.computerCompanyDBDataSet);

                MessageBox.Show("Данные успешно сохранены!", "Успех",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка при сохранении данных: " + ex.Message,
                    "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }

        private void categoriesBindingNavigatorSaveItem_Click(object sender,
            EventArgs e)
        {
            SaveCategoriesData();
        }

        private void CategoriesForm_Load(object sender, EventArgs e)
        {
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Categories". При необходимости она может быть перемещена
            или удалена.

            this.categoriesTableAdapter.Fill(this.computerCompanyDBDataSet.Categories);

            // Привязка элементов управления к полям данных
            textBoxCategoryName.DataBindings.Add("Text", categoriesBindingSource,
                "CategoryName", true, DataSourceUpdateMode.OnPropertyChanged);
        }
    }
}

```

```

        textBoxDescription.DataBindings.Add("Text", categoriesBindingSource,
        "Description", true, DataSourceUpdateMode.OnPropertyChanged);

        categoriesDataGridView.Columns[0].Visible = false;
        categoriesDataGridView.Columns[1].HeaderText = "Название категории";
        categoriesDataGridView.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
        categoriesDataGridView.Columns[2].HeaderText = "Описание";
        categoriesDataGridView.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;
    }

    private void btFirst_Click(object sender, EventArgs e)
    {
        categoriesBindingSource.MoveFirst();
    }

    private void btLast_Click(object sender, EventArgs e)
    {
        categoriesBindingSource.MoveLast();
    }

    private void btPrevious_Click(object sender, EventArgs e)
    {
        categoriesBindingSource.MovePrevious();
    }

    private void btNext_Click(object sender, EventArgs e)
    {
        categoriesBindingSource.MoveNext();
    }

    private void btAdd_Click(object sender, EventArgs e)
    {
        addCategoriesForm = new AddCategoriesForm();
        if (addCategoriesForm.ShowDialog() == DialogResult.OK)
        {
            // Получение данных из формы
            string categoryName = addCategoriesForm.CategoryName;
            string description = addCategoriesForm.Description;

            DataRow newRow = computerCompanyDBDataSet.Categories.NewRow();
            newRow["CategoryName"] = categoryName;
            newRow["Description"] = description;
            computerCompanyDBDataSet.Categories.Rows.Add(newRow);

            categoriesBindingSource.ResetBindings(false);
        }
    }

    private void btRemove_Click(object sender, EventArgs e)
    {
        categoriesBindingSource.RemoveCurrent();
    }

    private void btSave_Click(object sender, EventArgs e)
    {
        categoriesBindingNavigatorSaveItem_Click(sender, e);
    }

    private void btCancel_Click(object sender, EventArgs e)
    {
        computerCompanyDBDataSet.Categories.RejectChanges();
        categoriesDataGridView.Refresh();
    }

```

```

    }

e) private void CategoriesForm_FormClosing(object sender, FormClosingEventArgs
    {
        if (promptOnClose)
        {
            DialogResult result = MessageBox.Show(
                "Вы хотите обновить список категорий?",
                "Обновление данных",
                MessageBoxButtons.YesNo,
                MessageBoxIcon.Question
            );

            if (result == DialogResult.Yes)
            {
                SaveCategoriesData();

                if (this.Owner is ComponentsForm mainComponents)
                {
                    if (mainComponents.comboBoxCategoryId.Items.Count > 0)
                    {
                        object prevCategoryId =
mainComponents.comboBoxCategoryId.SelectedValue;

mainComponents.categoriesTableAdapter.Fill(mainComponents.computerCompanyDBDataSet.
Categories);

                        if (prevCategoryId != null &&
mainComponents.comboBoxCategoryId.Items.Cast<DataRowView>().Any(item =>
item.Row["CategoryID"].Equals(prevCategoryId)))
                        {
                            mainComponents.comboBoxCategoryId.SelectedValue =
prevCategoryId;
                        }
                        else
                        {
                            mainComponents.comboBoxCategoryId.SelectedIndex =
0;
                        }
                    }
                }
                else if (this.Owner is AddComponentsForm addComponents)
                {
                    if (addComponents.comboBoxCategoryId.Items.Count > 0)
                    {
                        object prevCategoryId =
addComponents.comboBoxCategoryId.SelectedValue;

addComponents.categoriesTableAdapter.Fill(addComponents.computerCompanyDBDataSet.Ca
tegories);

                        if (prevCategoryId != null &&
addComponents.comboBoxCategoryId.Items.Cast<DataRowView>().Any(item =>
item.Row["CategoryID"].Equals(prevCategoryId)))
                        {
                            addComponents.comboBoxCategoryId.SelectedValue =
prevCategoryId;
                        }
                        else
                        {
                            addComponents.comboBoxCategoryId.SelectedIndex = 0;
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
    }
}

private void textBox_Validating(object sender, CancelEventArgs e)
{
    TextBox textBox = sender as TextBox;

    if (string.IsNullOrEmpty(textBox.Text))
    {
        MessageBox.Show($"Поле '{textBox.Tag}' не может быть пустым.",
            "Ошибка валидации", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        categoriesBindingSource.CancelEdit();
        e.Cancel = true; // Отменяем событие, чтобы фокус остался на поле
    }
}
}
}

```

Форма ComponentsForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class ComponentsForm : Form
    {
        private AddComponentsForm addComponentsForm;
        private bool promptOnClose; // Флаг для показа MessageBox
        public ComponentsForm(bool promptOnClose = false)
        {
            InitializeComponent();
            this.promptOnClose = promptOnClose;
        }

        private void SaveComponentsData()
        {
            try
            {
                this.Validate();
                this.componentsBindingSource.EndEdit();
                this.tableAdapterManager.UpdateAll(this.computerCompanyDBDataSet);

                MessageBox.Show("Данные успешно сохранены!", "Успех",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка при сохранении данных: " + ex.Message,
                    "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
}

```

```

        private void componentsBindingNavigatorSaveItem_Click(object sender,
EventArgs e)
        {
            SaveComponentsData();
        }

        private void ComponentsForm_Load(object sender, EventArgs e)
        {
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Categories". При необходимости она может быть перемещена
            или удалена.

            this.categoriesTableAdapter.Fill(this.computerCompanyDBDataSet.Categories);
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Components". При необходимости она может быть перемещена
            или удалена.

            this.componentsTableAdapter.Fill(this.computerCompanyDBDataSet.Components);

            // Привязка элементов управления к полям данных
            textBoxComponentName.DataBindings.Add("Text",
            fKComponentCateg6EF57B66BindingSource, "ComponentName", true,
            DataSourceUpdateMode.OnPropertyChanged);
            textBoxPrice.DataBindings.Add("Text",
            fKComponentCateg6EF57B66BindingSource, "Price", true,
            DataSourceUpdateMode.OnPropertyChanged);

            // Удаляем все колонки перед добавлением новых
            componentsDataGridView.Columns.Clear();
            componentsDataGridView.AutoGenerateColumns = false;

            // Добавляем оригинальный (скрытый) столбец CategoryID
            DataGridViewTextBoxColumn hiddenCategoryColumn = new
            DataGridViewTextBoxColumn
            {
                DataPropertyName = "CategoryID",
                Name = "CategoryID"
            };
            componentsDataGridView.Columns.Add(hiddenCategoryColumn);
            hiddenCategoryColumn.Visible = false;

            componentsDataGridView.Columns.Add(new DataGridViewTextBoxColumn
            {
                DataPropertyName = "ComponentName",
                HeaderText = "Название комплектующего"
            });

            componentsDataGridView.Columns.Add(new DataGridViewTextBoxColumn
            {
                DataPropertyName = "Price",
                HeaderText = "Цена"
            });

            DataGridViewTextBoxColumn categoryNameColumn = new
            DataGridViewTextBoxColumn
            {
                Name = "CategoryName",
                HeaderText = "Категория",
                ReadOnly = true // Только для чтения
            };
            componentsDataGridView.Columns.Add(categoryNameColumn);
            categoryNameColumn.Visible = false;

```

```

        componentsDataGridView.CellFormatting +=
ComponentsDataGridView_CellFormatting;
    }

    private void btFirst_Click(object sender, EventArgs e)
    {
        fKComponentCateg6EF57B66BindingSource.MoveFirst();
    }

    private void btLast_Click(object sender, EventArgs e)
    {
        fKComponentCateg6EF57B66BindingSource.MoveLast();
    }

    private void btPrevious_Click(object sender, EventArgs e)
    {
        fKComponentCateg6EF57B66BindingSource.MovePrevious();
    }

    private void btNext_Click(object sender, EventArgs e)
    {
        fKComponentCateg6EF57B66BindingSource.MoveNext();
    }

    private void btAdd_Click(object sender, EventArgs e)
    {
        addComponentsForm = new AddComponentsForm();
        if (addComponentsForm.ShowDialog() == DialogResult.OK)
        {
            string componentName = addComponentsForm.ComponentName;
            decimal price = addComponentsForm.Price;
            int categoryId = addComponentsForm.SelectedCategoryId;

            DataRow newRow = computerCompanyDBDataSet.Components.NewRow();
            newRow["ComponentName"] = componentName;
            newRow["Price"] = price;
            newRow["CategoryID"] = categoryId;
            computerCompanyDBDataSet.Components.Rows.Add(newRow);

            componentsBindingSource.ResetBindings(false);

            comboBoxCategoryId.SelectedValue = categoryId;
        }
    }

    private void btRemove_Click(object sender, EventArgs e)
    {
        fKComponentCateg6EF57B66BindingSource.RemoveCurrent();
    }

    private void btSave_Click(object sender, EventArgs e)
    {
        SaveComponentsData();
    }

    private void btCancel_Click(object sender, EventArgs e)
    {
        computerCompanyDBDataSet.Components.RejectChanges();
        componentsDataGridView.Refresh();
    }

    private void ComponentsDataGridView_CellFormatting(object sender,
DataGridViewCellFormattingEventArgs e)
    {

```

```

        if (componentsDataGridView.Columns[e.ColumnIndex].Name ==
"CategoryName")
        {
            var row = componentsDataGridView.Rows[e.RowIndex];

            if (row.Cells["CategoryID"].Value != null &&
int.TryParse(row.Cells["CategoryID"].Value.ToString(), out int categoryID))
            {
                DataRow[] foundRows =
computerCompanyDBDataSet.Categories.Select($"CategoryID = {categoryID}");
                if (foundRows.Length > 0)
                {
                    e.Value = foundRows[0]["CategoryName"].ToString();
                    e.FormattingApplied = true;
                }
            }
        }
    }

e) private void ComponentsForm_FormClosing(object sender, FormClosingEventArgs
    {
        if (promptOnClose)
        {
            DialogResult result = MessageBox.Show(
                "Вы хотите обновить список комплектующих?",
                "Обновление данных",
                MessageBoxButtons.YesNo,
                MessageBoxIcon.Question
            );

            if (result == DialogResult.Yes)
            {
                SaveComponentsData();

                if (this.Owner is PurchaseDetailsForm main)
                {
                    if (main.comboBoxComponentId.Items.Count > 0)
                    {
                        object prevComponentId =
main.comboBoxComponentId.SelectedValue;

main.componentsTableAdapter.Fill(main.computerCompanyDBDataSet.Components);

                        if (prevComponentId != null &&
main.comboBoxComponentId.Items.Cast<DataRowView>().Any(item =>
item.Row["ComponentID"].Equals(prevComponentId)))
                        {
                            main.comboBoxComponentId.SelectedValue =
prevComponentId;
                        }
                        else
                        {
                            main.comboBoxComponentId.SelectedIndex = 0;
                        }
                    }
                }
                else if (this.Owner is AddPurchaseDetailsForm
addPurchaseDetails)
                {
                    if (addPurchaseDetails.comboBoxComponentId.Items.Count > 0)
                    {
                        object prevComponentId =
addPurchaseDetails.comboBoxComponentId.SelectedValue;

```

```
addPurchaseDetails.componentsTableAdapter.Fill(addPurchaseDetails.computerCompanyDB
DataSet.Components);
```

```
        if (prevComponentId != null &&
addPurchaseDetails.comboBoxComponentId.Items.Cast<DataRowView>().Any(item =>
item.Row["ComponentID"].Equals(prevComponentId)))
        {
```

```
addPurchaseDetails.comboBoxComponentId.SelectedValue = prevComponentId;
        }
        else
        {
```

```
addPurchaseDetails.comboBoxComponentId.SelectedIndex = 0;
        }
    }
```

```
    }
}
}
```

```
private void buttonCategories_Click(object sender, EventArgs e)
{
    CategoriesForm categoriesForm = new CategoriesForm(true); // Включаем
    MessageBox categoriesForm.Owner = this;
    categoriesForm.ShowDialog();
}
```

```
private void textBox_Validating(object sender, CancelEventArgs e)
{
    TextBox textBox = sender as TextBox;

    if (textBox.Tag.ToString() == "Название комплектующего")
    {
        // Валидация для названия
        if (string.IsNullOrEmpty(textBox.Text))
        {
            MessageBox.Show($"Поле '{textBox.Tag}' не может быть пустым.",
"Ошибка валидации", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            fKComponentCateg6EF57B66BindingSource.CancelEdit();
            e.Cancel = true;
        }
    }
    else if (textBox.Tag.ToString() == "Цена")
    {
        // Валидация для цены
        if (!decimal.TryParse(textBox.Text, out decimal price) || price <=
0)
        {
            MessageBox.Show($"Поле '{textBox.Tag}' должно содержать
положительное число.", "Ошибка валидации", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
            fKComponentCateg6EF57B66BindingSource.CancelEdit();
            e.Cancel = true;
        }
    }
}
}
```

Форма SuppliersForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class SuppliersForm : Form
    {
        private AddSuppliersForm addSuppliersForm;
        private bool promptOnClose; // Флаг для показа MessageBox
        public SuppliersForm(bool promptOnClose = false)
        {
            InitializeComponent();
            this.promptOnClose = promptOnClose;
        }

        private void SaveSuppliersData()
        {
            try
            {
                this.Validate();
                this.suppliersBindingSource.EndEdit();
                this.tableAdapterManager.UpdateAll(this.computerCompanyDBDataSet);

                MessageBox.Show("Данные успешно сохранены!", "Успех",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка при сохранении данных: " + ex.Message,
                    "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }

        private void suppliersBindingNavigatorSaveItem_Click(object sender,
            EventArgs e)
        {
            SaveSuppliersData();
        }

        private void SuppliersForm_Load(object sender, EventArgs e)
        {
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Suppliers". При необходимости она может быть перемещена
            или удалена.

            this.suppliersTableAdapter.Fill(this.computerCompanyDBDataSet.Suppliers);

            textBoxSupplierName.DataBindings.Add("Text", suppliersBindingSource,
                "SupplierName", true, DataSourceUpdateMode.OnPropertyChanged);
            textBoxContactInfo.DataBindings.Add("Text", suppliersBindingSource,
                "ContactInfo", true, DataSourceUpdateMode.OnPropertyChanged);
            textBoxAddress.DataBindings.Add("Text", suppliersBindingSource,
                "Address", true, DataSourceUpdateMode.OnPropertyChanged);
        }
    }
}

```

```

        suppliersDataGridView.Columns[0].Visible = false;
        suppliersDataGridView.Columns[1].HeaderText = "Имя поставщика";
        suppliersDataGridView.Columns[2].HeaderText = "Контакты";
        suppliersDataGridView.Columns[3].HeaderText = "Адрес";
        suppliersDataGridView.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells; // Столбец "Имя поставщика" подстраивается
под содержимое
        suppliersDataGridView.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill; // Остальные заполняют оставшееся пространство
        suppliersDataGridView.Columns[3].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
    }

    private void btFirst_Click(object sender, EventArgs e)
    {
        suppliersBindingSource.MoveFirst();
    }

    private void btLast_Click(object sender, EventArgs e)
    {
        suppliersBindingSource.MoveLast();
    }

    private void btPrevious_Click(object sender, EventArgs e)
    {
        suppliersBindingSource.MovePrevious();
    }

    private void btNext_Click(object sender, EventArgs e)
    {
        suppliersBindingSource.MoveNext();
    }

    private void btAdd_Click(object sender, EventArgs e)
    {
        addSuppliersForm = new AddSuppliersForm();
        if (addSuppliersForm.ShowDialog() == DialogResult.OK)
        {
            // Получение данных из формы
            string supplierName = addSuppliersForm.SupplierName;
            string contacts = addSuppliersForm.Contacts;
            string address = addSuppliersForm.Address;

            DataRow newRow = computerCompanyDBDataSet.Suppliers.NewRow();
            newRow["SupplierName"] = supplierName;
            newRow["ContactInfo"] = contacts;
            newRow["Address"] = address;
            computerCompanyDBDataSet.Suppliers.Rows.Add(newRow);

            suppliersBindingSource.ResetBindings(false);
        }
    }

    private void btRemove_Click(object sender, EventArgs e)
    {
        suppliersBindingSource.RemoveCurrent();
    }

    private void btSave_Click(object sender, EventArgs e)
    {
        SaveSuppliersData();
    }

```

```

private void btCancel_Click(object sender, EventArgs e)
{
    computerCompanyDBDataSet.Suppliers.RejectChanges();
    suppliersDataGridView.Refresh();
}

e) private void SuppliersForm_FormClosing(object sender, FormClosingEventArgs
{
    if (promptOnClose)
    {
        DialogResult result = MessageBox.Show(
            "Вы хотите обновить список поставщиков?",
            "Обновление данных",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question
        );

        if (result == DialogResult.Yes)
        {
            SaveSuppliersData();

            if (this.Owner is PurchasesForm purchaseForm)
            {
                if (purchaseForm.comboBoxSupplierId.Items.Count > 0)
                {
                    object prevSupplierId =
purchaseForm.comboBoxSupplierId.SelectedValue;

purchaseForm.suppliersTableAdapter.Fill(purchaseForm.computerCompanyDBDataSet.Suppl
iers);

                    if (prevSupplierId != null &&
purchaseForm.comboBoxSupplierId.Items.Cast<DataRowView>().Any(item =>
item.Row["SupplierID"].Equals(prevSupplierId)))
                    {
                        purchaseForm.comboBoxSupplierId.SelectedValue =
prevSupplierId;
                    }
                    else
                    {
                        purchaseForm.comboBoxSupplierId.SelectedIndex = 0;
                    }
                }
            }
            else if (this.Owner is AddPurchasesForm addPurchase)
            {
                if (addPurchase.comboBoxSupplierId.Items.Count > 0)
                {
                    object prevSupplierId =
addPurchase.comboBoxSupplierId.SelectedValue;

addPurchase.suppliersTableAdapter.Fill(addPurchase.computerCompanyDBDataSet.Supplie
rs);

                    if (prevSupplierId != null &&
addPurchase.comboBoxSupplierId.Items.Cast<DataRowView>().Any(item =>
item.Row["SupplierID"].Equals(prevSupplierId)))
                    {
                        addPurchase.comboBoxSupplierId.SelectedValue =
prevSupplierId;
                    }
                    else
                    {

```

```

        addPurchase.comboBoxSupplierId.SelectedIndex = 0;
    }
}
else if (this.Owner is MainForm mainForm)
{
    if (mainForm.comboBoxSupplier.Items.Count > 0)
    {
        object prevSupplierId =
mainForm.comboBoxSupplier.SelectedValue;

mainForm.suppliersTableAdapter.Fill(mainForm.computerCompanyDBDataSet.Suppliers);

        if (prevSupplierId != null &&
mainForm.comboBoxSupplier.Items.Cast<DataRowView>().Any(item =>
item.Row["SupplierID"].Equals(prevSupplierId)))
        {
            mainForm.comboBoxSupplier.SelectedValue =
prevSupplierId;
        }
        else
        {
            mainForm.comboBoxSupplier.SelectedIndex = 0;
        }
    }
}
}

private void textBox_Validating(object sender, CancelEventArgs e)
{
    TextBox textBox = sender as TextBox;

    if (textBox.Tag.ToString() == "Имя поставщика" ||
        textBox.Tag.ToString() == "Контакты" ||
        textBox.Tag.ToString() == "Адрес")
    {
        // Проверка на пустое значение
        if (string.IsNullOrEmpty(textBox.Text))
        {
            MessageBox.Show($"Поле '{textBox.Tag}' не может быть пустым.",
"Ошибка валидации", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            suppliersBindingSource.CancelEdit();
            e.Cancel = true; // Отменяем событие, чтобы фокус остался на
поле
        }
    }
}
}

```

Форма PurchasesForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class PurchasesForm : Form
    {
        private AddPurchasesForm addPurchaseForm;
        private bool promptOnClose; // Флаг для показа MessageBox
        public PurchasesForm(bool promptOnClose = false) // По умолчанию false
        {
            InitializeComponent();
            this.promptOnClose = promptOnClose;
        }

        private void SavePurchases()
        {
            try
            {
                this.Validate();
                this.purchasesBindingSource.EndEdit();
                this.tableAdapterManager.UpdateAll(this.computerCompanyDBDataSet);

                MessageBox.Show("Данные успешно сохранены!", "Успех",
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка при сохранении данных: " + ex.Message,
                    "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }

        private void purchasesBindingNavigatorSaveItem_Click(object sender,
            EventArgs e)
        {
            SavePurchases();
        }

        private void PurchasesForm_Load(object sender, EventArgs e)
        {
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Suppliers". При необходимости она может быть перемещена
            или удалена.

            this.suppliersTableAdapter.Fill(this.computerCompanyDBDataSet.Suppliers);
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Purchases". При необходимости она может быть перемещена
            или удалена.

            this.purchasesTableAdapter.Fill(this.computerCompanyDBDataSet.Purchases);

            // Привязка элементов управления к полям данных

```

```

        textBoxPurchaseId.DataBindings.Add("Text", purchasesBindingSource,
"PurchaseID", true, DataSourceUpdateMode.Never);
        comboBoxSupplierId.DataBindings.Add("SelectedValue",
purchasesBindingSource, "SupplierID", true,
DataSourceUpdateMode.OnPropertyChanged);
        dateTimePickerPurchaseDate.DataBindings.Add("Value",
purchasesBindingSource, "PurchaseDate", true,
DataSourceUpdateMode.OnPropertyChanged);
        textBoxPurchaseReason.DataBindings.Add("Text", purchasesBindingSource,
"PurchaseReason", true, DataSourceUpdateMode.OnPropertyChanged);

        // Удаляем все колонки перед добавлением новых
purchasesDataGridView.Columns.Clear();
purchasesDataGridView.AutoGenerateColumns = false;

        // Добавляем оригинальный (скрытый) столбец SupplierID
DataGridViewTextBoxColumn hiddenSupplierColumn = new
DataGridViewTextBoxColumn
{
    DataPropertyName = "SupplierID",
    Name = "SupplierID"
};
purchasesDataGridView.Columns.Add(hiddenSupplierColumn);
hiddenSupplierColumn.Visible = false;

        DataGridViewTextBoxColumn supplierNameColumn = new
DataGridViewTextBoxColumn
{
    Name = "SupplierName",
    HeaderText = "Имя поставщика",
    ReadOnly = true
};
purchasesDataGridView.Columns.Add(supplierNameColumn);

        DataGridViewTextBoxColumn purchaseDateColumn = new
DataGridViewTextBoxColumn
{
    DataPropertyName = "PurchaseDate",
    HeaderText = "Дата покупки"
};
purchasesDataGridView.Columns.Add(purchaseDateColumn);

        DataGridViewTextBoxColumn purchaseReasonColumn = new
DataGridViewTextBoxColumn
{
    DataPropertyName = "PurchaseReason",
    HeaderText = "Причина закупки"
};
purchasesDataGridView.Columns.Add(purchaseReasonColumn);

        supplierNameColumn.AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells; // "Имя поставщика" подстраивается под
содержимое
        purchaseDateColumn.AutoSizeMode = DataGridViewAutoSizeColumnMode.Fill;
// Остальные равномерно занимают оставшуюся ширину
        purchaseReasonColumn.AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
        purchasesDataGridView.CellFormatting +=
PurchasesDataGridView_CellFormatting;
    }

```

```

        private void PurchasesDataGridView_CellFormatting(object sender,
DataGridViewCellFormattingEventArgs e)
        {
            if (purchasesDataGridView.Columns[e.ColumnIndex].Name ==
"SupplierName")
            {
                var row = purchasesDataGridView.Rows[e.RowIndex];

                if (row.Cells["SupplierID"].Value != null &&
int.TryParse(row.Cells["SupplierID"].Value.ToString(), out int supplierID))
                {
                    DataRow[] foundRows =
computerCompanyDBDataSet.Suppliers.Select($"SupplierID = {supplierID}");
                    if (foundRows.Length > 0)
                    {
                        e.Value = foundRows[0]["SupplierName"].ToString();
                        e.FormattingApplied = true;
                    }
                }
            }
        }

        private void btFirst_Click(object sender, EventArgs e)
        {
            purchasesBindingSource.MoveFirst();
        }

        private void btLast_Click(object sender, EventArgs e)
        {
            purchasesBindingSource.MoveLast();
        }

        private void btPrevious_Click(object sender, EventArgs e)
        {
            purchasesBindingSource.MovePrevious();
        }

        private void btNext_Click(object sender, EventArgs e)
        {
            purchasesBindingSource.MoveNext();
        }

        private void btAdd_Click(object sender, EventArgs e)
        {
            addPurchaseForm = new AddPurchasesForm();
            if (addPurchaseForm.ShowDialog() == DialogResult.OK)
            {
                // Получение данных из формы
                int supplierId = addPurchaseForm.SelectedSupplierId;
                DateTime purchaseDate = addPurchaseForm.PurchaseDate;
                string purchaseReason = addPurchaseForm.PurchaseReason;

                DataRow newRow = computerCompanyDBDataSet.Purchases.NewRow();
                newRow["SupplierId"] = supplierId;
                newRow["PurchaseDate"] = purchaseDate;
                newRow["PurchaseReason"] = purchaseReason;
                computerCompanyDBDataSet.Purchases.Rows.Add(newRow);

                purchasesBindingSource.ResetBindings(false);
            }
        }

        private void btRemove_Click(object sender, EventArgs e)
        {

```

```

        purchasesBindingSource.RemoveCurrent();
    }

    private void btSave_Click(object sender, EventArgs e)
    {
        SavePurchases();
    }

    private void btCancel_Click(object sender, EventArgs e)
    {
        computerCompanyDBDataSet.Purchases.RejectChanges();
        purchasesDataGridView.Refresh();
    }
    private void PurchasesForm_FormClosing(object sender, FormClosingEventArgs
e)
    {
        if (promptOnClose)
        {
            DialogResult result = MessageBox.Show(
                "Вы хотите обновить список покупок?",
                "Обновление данных",
                MessageBoxButtons.YesNo,
                MessageBoxIcon.Question
            );

            if (result == DialogResult.Yes)
            {
                SavePurchases();

                if (this.Owner is PurchaseDetailsForm purchaseDetailsForm)
                {
                    if (purchaseDetailsForm.comboBoxPurchaseId.Items.Count > 0)
                    {
                        object prevPurchaseId =
purchaseDetailsForm.comboBoxPurchaseId.SelectedValue;

purchaseDetailsForm.purchasesTableAdapter.Fill(purchaseDetailsForm.computerCompanyD
BDataSet.Purchases);

                        if (prevPurchaseId != null &&
purchaseDetailsForm.comboBoxPurchaseId.Items.Cast<DataRowView>().Any(item =>
item.Row["PurchaseID"].Equals(prevPurchaseId)))
                        {

purchaseDetailsForm.comboBoxPurchaseId.SelectedValue = prevPurchaseId;
                        }
                        else
                        {

purchaseDetailsForm.comboBoxPurchaseId.SelectedIndex = 0;
                        }
                    }
                }
            }
            else if (this.Owner is AddPurchaseDetailsForm
addPurchaseDetails)
            {
                if (addPurchaseDetails.comboBoxPurchaseId.Items.Count > 0)
                {
                    object prevPurchaseId =
addPurchaseDetails.comboBoxPurchaseId.SelectedValue;

addPurchaseDetails.purchasesTableAdapter.Fill(addPurchaseDetails.computerCompanyDBD
ataSet.Purchases);

```

```

        if (prevPurchaseId != null &&
addPurchaseDetails.comboBoxPurchaseId.Items.Cast<DataRowView>().Any(item =>
item.Row["PurchaseID"].Equals(prevPurchaseId)))
        {
            addPurchaseDetails.comboBoxPurchaseId.SelectedValue
= prevPurchaseId;
        }
        else
        {
            addPurchaseDetails.comboBoxPurchaseId.SelectedIndex
= 0;
        }
    }
}

private void buttonSuppliers_Click(object sender, EventArgs e)
{
    SuppliersForm suppliersForm = new SuppliersForm(true); // Включаем
    suppliersForm.Owner = this;
    suppliersForm.ShowDialog();
}

private void textBox_Validating(object sender, CancelEventArgs e)
{
    TextBox textBox = sender as TextBox;

    if (string.IsNullOrEmpty(textBox.Text))
    {
        MessageBox.Show($"Поле '{textBox.Tag}' не может быть пустым.",
"Ошибка валидации", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        purchasesBindingSource.CancelEdit();
        e.Cancel = true; // Отменяем событие, чтобы фокус остался на поле
    }
}
}

```

Форма PurchaseDetailsForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class PurchaseDetailsForm : Form
    {
        private AddPurchaseDetailsForm addPurchaseDetailsForm;
        private CheckForm checkForm;
        public PurchaseDetailsForm()
        {
            InitializeComponent();
        }
    }
}

```

```

private void SavePurchaseDetails()
{
    try
    {
        this.Validate();
        this.purchaseDetailsBindingSource.EndEdit();
        this.tableAdapterManager.UpdateAll(this.computerCompanyDBDataSet);

        MessageBox.Show("Данные успешно сохранены!", "Успех",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    catch (Exception ex)
    {
        MessageBox.Show("Ошибка при сохранении данных: " + ex.Message,
        "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void purchaseDetailsBindingNavigatorSaveItem_Click(object sender,
EventArgs e)
{
    SavePurchaseDetails();
}

private void PurchaseDetailsForm_Load(object sender, EventArgs e)
{
    // TODO: данная строка кода позволяет загрузить данные в таблицу
    "computerCompanyDBDataSet.Components". При необходимости она может быть перемещена
    или удалена.

    this.componentsTableAdapter.Fill(this.computerCompanyDBDataSet.Components);
    // TODO: данная строка кода позволяет загрузить данные в таблицу
    "computerCompanyDBDataSet.Purchases". При необходимости она может быть перемещена
    или удалена.

    this.purchasesTableAdapter.Fill(this.computerCompanyDBDataSet.Purchases);
    // TODO: данная строка кода позволяет загрузить данные в таблицу
    "computerCompanyDBDataSet.PurchaseDetails". При необходимости она может быть
    перемещена или удалена.

    this.purchaseDetailsTableAdapter.Fill(this.computerCompanyDBDataSet.PurchaseDetails
    );

    // Привязка элементов управления к полям данных
    comboBoxPurchaseId.DataBindings.Add("SelectedValue",
    purchaseDetailsBindingSource, "PurchaseID", true,
    DataSourceUpdateMode.OnPropertyChanged);
    comboBoxComponentId.DataBindings.Add("SelectedValue",
    fKPurchaseDPurch76969D2EBindingSource, "ComponentID", true,
    DataSourceUpdateMode.OnPropertyChanged);
    textBoxQuantity.DataBindings.Add("Text",
    fKPurchaseDPurch76969D2EBindingSource, "Quantity", true,
    DataSourceUpdateMode.OnPropertyChanged);
    textBoxUnitPrice.DataBindings.Add("Text",
    fKPurchaseDPurch76969D2EBindingSource, "UnitPrice", true,
    DataSourceUpdateMode.Never);

    purchaseDetailsDataGridView.Columns.Clear();
    purchaseDetailsDataGridView.AutoGenerateColumns = false;

    // Добавляем оригинальный (скрытый) столбец ComponentID
    DataGridViewTextBoxColumn hiddenComponentColumn = new
    DataGridViewTextBoxColumn

```

```

    {
        DataPropertyName = "ComponentID",
        Name = "ComponentID"
    };
    purchaseDetailsDataGridView.Columns.Add(hiddenComponentColumn);
    hiddenComponentColumn.Visible = false;

    // Добавляем колонку для отображения ComponentName
    DataGridViewTextBoxColumn componentNameColumn = new
DataGridViewTextBoxColumn
    {
        Name = "ComponentName",
        HeaderText = "Название комплектующего",
        ReadOnly = true // Только для чтения
    };
    purchaseDetailsDataGridView.Columns.Add(componentNameColumn);

    // Добавляем остальные колонки
    DataGridViewTextBoxColumn quantityColumn = new
DataGridViewTextBoxColumn
    {
        DataPropertyName = "Quantity",
        HeaderText = "Количество"
    };
    purchaseDetailsDataGridView.Columns.Add(quantityColumn);

    DataGridViewTextBoxColumn unitPriceColumn = new
DataGridViewTextBoxColumn
    {
        DataPropertyName = "UnitPrice",
        HeaderText = "Цена за единицу"
    };
    purchaseDetailsDataGridView.Columns.Add(unitPriceColumn);

    componentNameColumn.AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells; // Столбец "Название комплектующего"
подстраивается под содержимое
    quantityColumn.AutoSizeMode = DataGridViewAutoSizeColumnMode.Fill; //
Остальные заполняют оставшееся пространство
    unitPriceColumn.AutoSizeMode = DataGridViewAutoSizeColumnMode.Fill;
    // Привязываем событие для подмены ComponentID -> ComponentName
    purchaseDetailsDataGridView.CellFormatting +=
PurchaseDetailsDataGridView_CellFormatting;
}

// Метод подмены ComponentID на ComponentName
private void PurchaseDetailsDataGridView_CellFormatting(object sender,
DataGridViewCellFormattingEventArgs e)
{
    if (purchaseDetailsDataGridView.Columns[e.ColumnIndex].Name ==
"ComponentName")
    {
        var row = purchaseDetailsDataGridView.Rows[e.RowIndex];

        if (row.Cells["ComponentID"].Value != null &&
int.TryParse(row.Cells["ComponentID"].Value.ToString(), out int componentID))
        {
            DataRow[] foundRows =
computerCompanyDBDataSet.Components.Select($"ComponentID = {componentID}");
            if (foundRows.Length > 0)
            {
                e.Value = foundRows[0]["ComponentName"].ToString();
                e.FormattingApplied = true;
            }
        }
    }
}

```

```

    }
}

private void btFirst_Click(object sender, EventArgs e)
{
    fKPurchaseDPurch76969D2EBindingSource.MoveFirst();
}

private void btLast_Click(object sender, EventArgs e)
{
    fKPurchaseDPurch76969D2EBindingSource.MoveLast();
}

private void btPrevious_Click(object sender, EventArgs e)
{
    fKPurchaseDPurch76969D2EBindingSource.MovePrevious();
}

private void btNext_Click(object sender, EventArgs e)
{
    fKPurchaseDPurch76969D2EBindingSource.MoveNext();
}

private void btAdd_Click(object sender, EventArgs e)
{
    addPurchaseDetailsForm = new AddPurchaseDetailsForm();
    if (addPurchaseDetailsForm.ShowDialog() == DialogResult.OK)
    {
        // Получение данных из формы
        int purchaseId = addPurchaseDetailsForm.SelectedPurchaseId;
        int componentId = addPurchaseDetailsForm.SelectedComponentId;
        int quantity = addPurchaseDetailsForm.Quantity;
        decimal unitPrice = addPurchaseDetailsForm.UnitPrice;

        DataRow newRow = computerCompanyDBDataSet.PurchaseDetails.NewRow();
        newRow["PurchaseId"] = purchaseId;
        newRow["ComponentId"] = componentId;
        newRow["Quantity"] = quantity;
        newRow["UnitPrice"] = unitPrice;
        computerCompanyDBDataSet.PurchaseDetails.Rows.Add(newRow);

        purchaseDetailsBindingSource.ResetBindings(false);

        comboBoxPurchaseId.SelectedValue = purchaseId;
    }
}

private void btRemove_Click(object sender, EventArgs e)
{
    fKPurchaseDPurch76969D2EBindingSource.RemoveCurrent();
}

private void btSave_Click(object sender, EventArgs e)
{
    SavePurchaseDetails();
}

private void btCancel_Click(object sender, EventArgs e)
{
    computerCompanyDBDataSet.PurchaseDetails.RejectChanges();
    purchaseDetailsDataGridView.Refresh();
}

```

```

private void buttonComponents_Click(object sender, EventArgs e)
{
    ComponentsForm componentsForm = new ComponentsForm(true); // Включаем
    componentsForm.Owner = this;
    componentsForm.ShowDialog();
}

private void buttonPurchases_Click(object sender, EventArgs e)
{
    PurchasesForm purchasesForm = new PurchasesForm(true); // Включаем
    purchasesForm.Owner = this;
    purchasesForm.ShowDialog();
}

private void buttonCheck_Click(object sender, EventArgs e)
{
    if (checkForm == null || checkForm.IsDisposed)
    {
        if (comboBoxPurchaseId.SelectedItem != null)
        {
            DataRowView selectedRow = comboBoxPurchaseId.SelectedItem as
            DataRowView;

            if (selectedRow != null)
            {
                int purchaseId = Convert.ToInt32(selectedRow[0]);

                checkForm = new CheckForm(purchaseId);
                checkForm.FormClosed += (s, args) => checkForm = null;
                checkForm.ShowDialog(this);
            }
            else
            {
                MessageBox.Show("Пожалуйста, выберите идентификатор покупки.");
            }
        }
    }

    private void textBox_Validating(object sender, CancelEventArgs e)
    {
        TextBox textBox = sender as TextBox;

        if (!int.TryParse(textBox.Text, out int quantity) || quantity < 0)
        {
            MessageBox.Show($"Поле '{textBox.Tag}' должно содержать
            положительное целое число.", "Ошибка валидации", MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
            fkPurchaseDPurch76969D2EBindingSource.CancelEdit();
            e.Cancel = true; // Отменяем событие, чтобы фокус остался на поле
        }
    }
}

```

Форма CheckForm

```

using Microsoft.Reporting.WinForms;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Diagnostics;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class CheckForm : Form
    {
        private int? PurchaseId;
        public CheckForm(int PurchaseId)
        {
            InitializeComponent();
            this.PurchaseId = PurchaseId;
        }

        private void CheckForm_Load(object sender, EventArgs e)
        {
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.GetPurchaseDetailsByPurchaseID". При необходимости она
            может быть перемещена или удалена.

            this.getPurchaseDetailsByPurchaseIDTableAdapter.Fill(this.computerCompanyDBDataSet.
            GetPurchaseDetailsByPurchaseID, PurchaseId);

            // Установка источника данных для ReportViewer
            ReportDataSource rds = new ReportDataSource("CheckDataSet",
            this.computerCompanyDBDataSet.GetPurchaseDetailsByPurchaseID as DataTable);
            reportViewer1.LocalReport.DataSources.Clear();
            reportViewer1.LocalReport.DataSources.Add(rds);
            this.reportViewer1.RefreshReport();
        }

        private void reportViewer_ReportExport(object sender, ReportExportEventArgs
e)
        {
            e.Cancel = true;

            string extension = this.GetRenderingExtension(e.Extension);

            SaveFileDialog saveFileDialog = new SaveFileDialog()
            {
                Title = "Сохранить как",
                CheckPathExists = true,
                InitialDirectory =
Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments),
                FileName = "Чек",
                Filter = e.Extension.LocalizedName + " (*" + extension + ")|*" +
extension + "|Все файлы(*.*)|*.*",
                FilterIndex = 0
            };

            if (saveFileDialog.ShowDialog(this) == DialogResult.OK)
            {

```

```

        this.reportViewer1.ExportDialog(e.Extension, e.DeviceInfo,
saveFileDialog.FileName);

        OpenFileWithPrompt(saveFileDialog.FileName);
    }
}

private string GetRenderingExtension(RenderingExtension extension)
{
    switch (extension.Name)
    {
        case "PDF":
            return ".pdf";
        case "EXCELOPENXML":
            return ".xlsx";
        case "WORDOPENXML":
            return ".docx";
        case "NULL":
            throw new NotImplementedException("Расширение не реализовано");
    }

    throw new NotImplementedException("Расширение не реализовано");
}

public static void OpenFileWithPrompt(string file)
{
    DialogResult result = MessageBox.Show(
        "Файл успешно сохранён.\nОткрыть его сейчас?",
        "Открыть файл",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question,
        MessageBoxDefaultButton.Button1);

    if (result == DialogResult.Yes)
    {
        Process.Start(new ProcessStartInfo(file) { UseShellExecute = true
    });
    }
}
}
}

```

Форма ReportForm

```

using System;
using Microsoft.Reporting.WinForms;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Net.Mail;
using System.Net;
using System.IO;
using System.Threading;
using System.Diagnostics;

namespace ComputerCompany
{
    public partial class ReportForm : Form
    {

```

```

private DateTime? startDate;
private DateTime? endDate;
private int? supplierId;
private string supplierName;
private bool reportFlag;

public ReportForm(DateTime? startDate, DateTime? endDate, int? supplierId,
string supplierName, bool reportFlag)
{
    InitializeComponent();
    this.startDate = startDate;
    this.endDate = endDate;
    this.supplierId = supplierId;
    this.supplierName = supplierName;
    this.reportFlag = reportFlag;
    this.computerCompanyDBDataSet.GetPurchaseDetailsForSuppliers.Reset();
}

private void ReportForm_Load(object sender, EventArgs e)
{
    reportViewerMain.LocalReport.DataSources.Clear();
    string reportDataSet;
    DataTable dataTable;

    if (reportFlag)
    {
        this.getFilteredPurchasesTableAdapter.Fill(this.computerCompanyDBDataSet.GetFiltere
dPurchases, supplierId, startDate, endDate);
        reportViewerMain.LocalReport.ReportEmbeddedResource =
"ComputerCompany.ReportTable.rdlc";
        reportDataSet = "FilteredDataSet";
        dataTable = computerCompanyDBDataSet.GetFilteredPurchases;
    }
    else
    {
        this.getPurchaseDetailsForSuppliersTableAdapter.Fill(this.computerCompanyDBDataSet.
GetPurchaseDetailsForSuppliers, startDate, endDate, supplierId);
        reportViewerMain.LocalReport.ReportEmbeddedResource =
"ComputerCompany.ReportOfAll.rdlc";
        reportDataSet = "PurchaseForSuppliersDataSet";
        dataTable =
computerCompanyDBDataSet.GetPurchaseDetailsForSuppliers;
    }

    string total;
    bool flag;

    if (supplierId == null)
    {
        total = startDate == null && endDate == null
            ? "Отчет по закупкам комплектующих за все время для всех
поставщиков"
            : $"Отчет по закупкам комплектующих с {startDate?.ToString("dd
MMM уууу")} до {endDate?.ToString("dd MMM уууу")} для всех поставщиков";
        flag = false;
    }
    else
    {
        total = startDate == null && endDate == null
            ? $"Отчет по закупкам комплектующих за все время для поставщика
{supplierName}"

```

```

        : $"Отчет по закупкам комплектующих с {startDate?.ToString("dd
MMM yyyy")} до {endDate?.ToString("dd MMM yyyy")} для поставщика {supplierName}";
        flag = true;
    }

    ReportDataSource rds = new ReportDataSource(reportDataSet, dataTable);
    ReportParameter totalText = new ReportParameter("TotalText", total);
    ReportParameter reportFlagParam = new
ReportParameter("ReportParameterFlag", flag.ToString());

    reportViewerMain.LocalReport.SetParameters(new ReportParameter[] {
totalText, reportFlagParam });
    reportViewerMain.LocalReport.DataSources.Add(rds);

    this.reportViewerMain.RefreshReport();
}

e) private void reportViewer_ReportExport(object sender, ReportExportEventArgs
{
    e.Cancel = true;

    string extension = this.GetRenderingExtension(e.Extension);

    SaveFileDialog saveFileDialog = new SaveFileDialog()
    {
        Title = "Сохранить как",
        CheckPathExists = true,
        InitialDirectory =
Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments),
        FileName = "Отчет",
        Filter = e.Extension.LocalizedName + " (*" + extension + ")|*" +
extension + "|Все файлы(*.*)|*.*",
        FilterIndex = 0
    };

    if (saveFileDialog.ShowDialog(this) == DialogResult.OK)
    {
        this.reportViewerMain.ExportDialog(e.Extension, e.DeviceInfo,
saveFileDialog.FileName);

        OpenFileWithPrompt(saveFileDialog.FileName);
    }
}

private string GetRenderingExtension(RenderingExtension extension)
{
    switch (extension.Name)
    {
        case "PDF":
            return ".pdf";
        case "EXCELOPENXML":
            return ".xlsx";
        case "WORDOPENXML":
            return ".docx";
        case "NULL":
            throw new NotImplementedException("Расширение не реализовано");
    }

    throw new NotImplementedException("Расширение не реализовано");
}

public static void OpenFileWithPrompt(string file)
{

```

```

        DialogResult result = MessageBox.Show(
            "Файл успешно сохранён.\nОткрыть его сейчас?",
            "Открыть файл",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question,
            MessageBoxDefaultButton.Button1);

        if (result == DialogResult.Yes)
        {
            Process.Start(new ProcessStartInfo(file) { UseShellExecute = true
        });
    }
}
}
}

```

Форма AddCategoriesForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class AddCategoriesForm : Form
    {
        public string CategoryName { get; private set; }
        public string Description { get; private set; }
        public AddCategoriesForm()
        {
            InitializeComponent();

            private void buttonAdd_Click(object sender, EventArgs e)
            {
                // Валидация ввода
                if (string.IsNullOrEmpty(textBoxCategoryName.Text) ||
                    string.IsNullOrEmpty(textBoxDescription.Text))
                {
                    MessageBox.Show("Пожалуйста, заполните все поля.", "Ошибка",
                        MessageBoxButtons.OK, MessageBoxIcon.Error);
                    return;
                }

                CategoryName = textBoxCategoryName.Text;
                Description = textBoxDescription.Text;

                this.DialogResult = DialogResult.OK;
                this.Close();
            }

            private void buttonCancel_Click(object sender, EventArgs e)
            {
                this.DialogResult = DialogResult.Cancel;
                this.Close();
            }

            private void AddCategoriesForm_Load(object sender, EventArgs e)

```

```

    {
        // TODO: данная строка кода позволяет загрузить данные в таблицу
        "computerCompanyDBDataSet.Categories". При необходимости она может быть перемещена
        или удалена.

        this.categoriesTableAdapter.Fill(this.computerCompanyDBDataSet.Categories);
    }
}

```

Форма AddComponentsForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class AddComponentsForm : Form
    {
        public int SelectedCategoryId { get; private set; }
        public string ComponentName { get; private set; }
        public decimal Price { get; private set; }

        public AddComponentsForm()
        {
            InitializeComponent();
        }

        private void AddComponentsForm_Load(object sender, EventArgs e)
        {
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Components". При необходимости она может быть перемещена
            или удалена.

            this.componentsTableAdapter.Fill(this.computerCompanyDBDataSet.Components);
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Categories". При необходимости она может быть перемещена
            или удалена.

            this.categoriesTableAdapter.Fill(this.computerCompanyDBDataSet.Categories);
        }

        private void buttonAdd_Click(object sender, EventArgs e)
        {
            // Валидация ввода
            if (string.IsNullOrEmpty(textBoxComponentName.Text) ||
                !decimal.TryParse(textBoxPrice.Text, out decimal price) ||
                comboBoxCategoryId.SelectedValue == null)
            {
                MessageBox.Show("Пожалуйста, заполните все поля корректно.",
                    "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return;
            }

            SelectedCategoryId = (int)comboBoxCategoryId.SelectedValue;
        }
    }
}

```

```

        ComponentName = textBoxComponentName.Text;
        Price = price;

        this.DialogResult = DialogResult.OK;
        this.Close();
    }

    private void buttonCancel_Click(object sender, EventArgs e)
    {
        this.DialogResult = DialogResult.Cancel;
        this.Close();
    }

    private void buttonCategories_Click(object sender, EventArgs e)
    {
        CategoriesForm categoriesForm = new CategoriesForm(true); // Включаем
        categoriesForm.Owner = this;
        categoriesForm.ShowDialog();
    }
}

```

Форма AddPurchaseDetailsForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class AddPurchaseDetailsForm : Form
    {
        public int SelectedPurchaseId { get; private set; }
        public int SelectedComponentId { get; private set; }
        public int Quantity { get; private set; }
        public decimal UnitPrice { get; private set; }

        public AddPurchaseDetailsForm()
        {
            InitializeComponent();
        }

        private void buttonPurchases_Click(object sender, EventArgs e)
        {
            PurchasesForm purchasesForm = new PurchasesForm(true); // Включаем
            purchasesForm.Owner = this;
            purchasesForm.ShowDialog();
        }

        private void buttonComponents_Click(object sender, EventArgs e)
        {
            ComponentsForm componentsForm = new ComponentsForm(true); // Включаем
            componentsForm.Owner = this;
            componentsForm.ShowDialog();
        }
    }
}

```

```

        private void AddPurchaseDetailsForm_Load(object sender, EventArgs e)
        {
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Components". При необходимости она может быть перемещена
            или удалена.

            this.componentsTableAdapter.Fill(this.computerCompanyDBDataSet.Components);
            // TODO: данная строка кода позволяет загрузить данные в таблицу
            "computerCompanyDBDataSet.Purchases". При необходимости она может быть перемещена
            или удалена.

            this.purchasesTableAdapter.Fill(this.computerCompanyDBDataSet.Purchases);
        }

        private void buttonAdd_Click(object sender, EventArgs e)
        {
            // Валидация ввода
            if (comboBoxPurchaseId.SelectedValue == null ||
                comboBoxComponentId.SelectedValue == null ||
                !int.TryParse(textBoxQuantity.Text, out int quantity))
            {
                MessageBox.Show("Пожалуйста, заполните все поля корректно.",
                    "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
                return;
            }

            SelectedPurchaseId = (int)comboBoxPurchaseId.SelectedValue;
            SelectedComponentId = (int)comboBoxComponentId.SelectedValue;
            Quantity = quantity;

            this.DialogResult = DialogResult.OK;
            this.Close();
        }

        private void buttonCancel_Click(object sender, EventArgs e)
        {
            this.DialogResult = DialogResult.Cancel;
            this.Close();
        }
    }
}

```

Форма AddPurchasesForm

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class AddPurchasesForm : Form
    {
        public int SelectedSupplierId { get; private set; }
        public DateTime PurchaseDate { get; private set; }
        public string PurchaseReason { get; private set; }
    }
}

```

```

public AddPurchasesForm()
{
    InitializeComponent();
}

private void buttonSuppliers_Click(object sender, EventArgs e)
{
    SuppliersForm suppliersForm = new SuppliersForm(true); // Включаем
    suppliersForm.Owner = this;
    suppliersForm.ShowDialog();
}

private void AddPurchaseForm_Load(object sender, EventArgs e)
{
    // TODO: данная строка кода позволяет загрузить данные в таблицу
    "computerCompanyDBDataSet.Suppliers". При необходимости она может быть перемещена
    или удалена.
    this.suppliersTableAdapter.Fill(this.computerCompanyDBDataSet.Suppliers);
}

private void buttonAdd_Click(object sender, EventArgs e)
{
    // Валидация ввода
    if (comboBoxSupplierId.SelectedValue == null ||
        string.IsNullOrEmpty(textBoxPurchaseReason.Text))
    {
        MessageBox.Show("Пожалуйста, заполните все поля корректно.",
            "Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }

    SelectedSupplierId = (int)comboBoxSupplierId.SelectedValue;
    PurchaseDate = dateTimePickerPurchaseDate.Value.Date;
    PurchaseReason = textBoxPurchaseReason.Text;

    this.DialogResult = DialogResult.OK;
    this.Close();
}

private void buttonCancel_Click(object sender, EventArgs e)
{
    this.DialogResult = DialogResult.Cancel;
    this.Close();
}
}

```

Форма AddSuppliersForm

```

using Microsoft.Reporting.Map.WebForms.BingMaps;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Diagnostics.Contracts;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

using System.Windows.Forms;

namespace ComputerCompany
{
    public partial class AddSuppliersForm : Form
    {
        public string SupplierName { get; private set; }
        public string Contacts { get; private set; }
        public string Address { get; private set; }

        public AddSuppliersForm()
        {
            InitializeComponent();

            private void suppliersBindingNavigatorSaveItem_Click(object sender,
EventArgs e)
            {
                this.Validate();
                this.suppliersBindingSource.EndEdit();
                this.tableAdapterManager.UpdateAll(this.computerCompanyDBDataSet);

            }

            private void AddSuppliersForm_Load(object sender, EventArgs e)
            {
                // TODO: данная строка кода позволяет загрузить данные в таблицу
"computerCompanyDBDataSet.Suppliers". При необходимости она может быть перемещена
или удалена.

this.suppliersTableAdapter.Fill(this.computerCompanyDBDataSet.Suppliers);

            }

            private void buttonAdd_Click(object sender, EventArgs e)
            {
                // Валидация ввода
                if (string.IsNullOrEmpty(textBoxSupplierName.Text) ||
                    string.IsNullOrEmpty(textBoxContactInfo.Text) ||
                    string.IsNullOrEmpty(textBoxAddress.Text))
                {
                    MessageBox.Show("Пожалуйста, заполните все поля.", "Ошибка",
MessageBoxButtons.OK, MessageBoxIcon.Error);
                    return;
                }

                SupplierName = textBoxSupplierName.Text;
                Contacts = textBoxContactInfo.Text;
                Address = textBoxAddress.Text;

                this.DialogResult = DialogResult.OK;
                this.Close();
            }

            private void buttonCancel_Click(object sender, EventArgs e)
            {
                this.DialogResult = DialogResult.Cancel;
                this.Close();
            }
        }
    }
}

```

Форма MainForm

```

using Microsoft.Reporting.WinForms;
using System;
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using System.Windows.Forms;
using System.Net;
using System.Drawing;

namespace ComputerCompany
{
    public partial class MainForm : Form
    {
        private PurchasesForm purchasesForm;
        private PurchaseDetailsForm purchaseDetailsForm;
        private ComponentsForm componentsForm;
        private SuppliersForm suppliersForm;
        private CategoriesForm categoriesForm;
        private ReportForm reportForm;
        public MainForm()
        {
            InitializeComponent();

        }

        private void RefreshData()
        {
            this.purchasesTableAdapter.Fill(computerCompanyDBDataSet.Purchases);

this.purchaseDetailsTableAdapter.Fill(computerCompanyDBDataSet.PurchaseDetails);
            this.componentsTableAdapter.Fill(computerCompanyDBDataSet.Components);
            this.categoriesTableAdapter.Fill(computerCompanyDBDataSet.Categories);
            this.suppliersTableAdapter.Fill(computerCompanyDBDataSet.Suppliers);
        }

        private void LoadData()
        {
            try
            {
                RefreshData();
                if
(!computerCompanyDBDataSet.Purchases.Columns.Contains("TotalQuantity"))
                {
                    computerCompanyDBDataSet.Purchases.Columns.Add("TotalQuantity",
typeof(int));
                }
                if
(!computerCompanyDBDataSet.Purchases.Columns.Contains("TotalPrice"))
                {
                    computerCompanyDBDataSet.Purchases.Columns.Add("TotalPrice",
typeof(decimal));
                }

                dataGridViewFull.DataSource = computerCompanyDBDataSet.Purchases;

                // Обновление данных для новых полей
                UpdateTotalColumns();

                // Настройка заголовков столбцов
                SetupDataGridView();

                // Скрытие столбцов

```

```

        dataGridViewFull.Columns["SupplierID"].Visible = false;
        dataGridViewFull.Columns["PurchaseID"].Visible = false;
        dataGridViewFull.CellFormatting += DataGridViewFull_CellFormatting;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Ошибка при загрузке данных: " + ex.Message);
    }
}

private void SetupDataGridView()
{
    // Настройка заголовков столбцов
    dataGridViewFull.Columns["PurchaseDate"].HeaderText = "Дата Закупки";
    dataGridViewFull.Columns["PurchaseReason"].HeaderText = "Причина
Закупки";
    dataGridViewFull.Columns["TotalQuantity"].HeaderText = "Общее
Количество";
    dataGridViewFull.Columns["TotalPrice"].HeaderText = "Общая Стоимость";

    // Добавление столбца для SupplierName
    if (!dataGridViewFull.Columns.Contains("SupplierName"))
    {
        DataGridViewTextBoxColumn supplierNameColumn = new
DataGridViewTextBoxColumn
        {
            Name = "SupplierName",
            HeaderText = "Имя Поставщика",
            ReadOnly = true
        };
        dataGridViewFull.Columns.Add(supplierNameColumn);
    }

    dataGridViewFull.Columns["SupplierName"].DisplayIndex = 1;

    dataGridViewFull.Columns["PurchaseDate"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;
    dataGridViewFull.Columns["PurchaseReason"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
    dataGridViewFull.Columns["TotalQuantity"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;
    dataGridViewFull.Columns["TotalPrice"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;

    dataGridViewFull.Refresh();
}

private void DataGridViewFull_CellFormatting(object sender,
DataGridViewCellFormattingEventArgs e)
{
    if (dataGridViewFull.Columns[e.ColumnIndex].Name == "SupplierName")
    {
        var row = dataGridViewFull.Rows[e.RowIndex];

        if (row.Cells["SupplierID"].Value != null &&
int.TryParse(row.Cells["SupplierID"].Value.ToString(), out int supplierID))
        {
            DataRow[] foundRows =
computerCompanyDBDataSet.Suppliers.Select($"SupplierID = {supplierID}");
            if (foundRows.Length > 0)
            {
                e.Value = foundRows[0]["SupplierName"].ToString();
                e.FormattingApplied = true;
            }
        }
    }
}

```

```

    }
}

private void UpdateTotalColumns()
{
    int grandTotalQuantity = 0;
    decimal grandTotalPrice = 0;

    foreach (DataRow purchaseRow in
computerCompanyDBDataSet.Purchases.Rows)
    {
        int purchaseId = (int)purchaseRow["PurchaseID"];
        var details =
computerCompanyDBDataSet.PurchaseDetails.AsEnumerable()
            .Where(d => d.Field<int>("PurchaseID") == purchaseId);

        int totalQuantity = details.Sum(d => d.Field<int>("Quantity"));
        decimal totalPrice = details.Sum(d => d.Field<int>("Quantity") *
d.Field<decimal>("UnitPrice"));

        purchaseRow["TotalQuantity"] = totalQuantity;
        purchaseRow["TotalPrice"] = totalPrice;

        grandTotalQuantity += totalQuantity;
        grandTotalPrice += totalPrice;
    }
    labelTotals.Text = $"Итого: Количество = {grandTotalQuantity}, Сумма =
{grandTotalPrice}";

    DataGridView totalRow =
(DataGridView)computerCompanyDBDataSet.Purchases.DefaultView.AddNew();
    totalRow["PurchaseReason"] = "Итого";
    totalRow["TotalQuantity"] = grandTotalQuantity;
    totalRow["TotalPrice"] = grandTotalPrice;
}

private void dateTimePicker_ValueChanged(object sender, EventArgs e)
{
    if (dateTimePicker1.Value > dateTimePicker2.Value)
    {
        MessageBox.Show("Дата начала не может быть больше даты окончания.",
"Ошибка", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        dateTimePicker1.Value = dateTimePicker2.Value; // Сбрасываем дату
        начала на дату окончания
    }
}

private void DataGridView_SelectionChanged(object sender, EventArgs e)
{
    if (dataGridViewFull.SelectedRows.Count > 0 &&
dataGridViewFull.SelectedRows[0].Index <
dataGridViewFull.Rows.Count - 1)
    {
        int selectedPurchaseId =
(int)dataGridViewFull.SelectedRows[0].Cells["PurchaseID"].Value;

        // Очистка ListBox
        listBoxItems.Items.Clear();

        // Получение деталей закупки
        var childItems =
computerCompanyDBDataSet.PurchaseDetails.AsEnumerable()

```

```

        .Where(row => row.Field<int>("PurchaseID") ==
selectedPurchaseId);

        decimal totalCost = 0;
        foreach (var item in childItems)
        {
            int componentId = item.Field<int>("ComponentID");
            int quantity = item.Field<int>("Quantity");
            decimal unitPrice = item.Field<decimal>("UnitPrice");
            decimal itemTotal = quantity * unitPrice;
            var componentRow =
computerCompanyDBDataSet.Components.AsEnumerable()
                .FirstOrDefault(c => c.Field<int>("ComponentID") ==
componentId);

            string componentName =
componentRow?.Field<string>("ComponentName") ?? "Неизвестное комплектующее";

            int categoryId = componentRow?.Field<int>("CategoryID") ?? 0;

            string categoryName =
computerCompanyDBDataSet.Categories.AsEnumerable()
                .FirstOrDefault(c => c.Field<int>("CategoryID") ==
categoryId)?.Field<string>("CategoryName") ?? "Неизвестная категория";

            // Добавление информации в ListBox
            listBoxItems.Items.Add($"Комплектующее: {componentName},
Категория: {categoryName}, Количество: {quantity}, Цена: {unitPrice:C}, Общая
стоимость: {itemTotal:C}");
            totalCost += itemTotal;
        }

        listBoxItems.Items.Add($"Общая стоимость товаров: {totalCost:C}");
    }
    else
    {
        listBoxItems.Items.Clear();
    }
}

private void buttonFilter_Click(object sender, EventArgs e)
{
    // Сохраняем выбранного поставщика перед фильтрацией
    int selectedSupplier = comboBoxSupplier.SelectedIndex;

    computerCompanyDBDataSet.Purchases.RejectChanges();
    RefreshData();

    comboBoxSupplier.SelectedIndex = selectedSupplier; // Восстанавливаем
выбранного поставщика
    string filter = "";

    if (!checkBoxAllSuppliers.Checked)
    {
        int supplierId =
(int)((DataRowView)comboBoxSupplier.SelectedItem)["SupplierID"];
        filter += $"SupplierID = {supplierId}";
    }

    if (!checkBoxAllTime.Checked)
    {
        if (dateTimePicker1.Value != null && dateTimePicker2.Value != null)
        {
            if (!string.IsNullOrEmpty(filter))

```

```

        {
            filter += " AND ";
        }
        filter += $"PurchaseDate >= '{dateTimePicker1.Value:yyyy-MM-dd}' AND PurchaseDate <= '{dateTimePicker2.Value:yyyy-MM-dd}'";
    }

    try
    {
        DataView view = new DataView(computerCompanyDBDataSet.Purchases);

        if (!string.IsNullOrEmpty(filter))
        {
            view.RowFilter = filter;
        }

        dataGridViewFull.DataSource = null; // Обновляем DataGridView
        dataGridViewFull.DataSource = view;

        // Добавление столбца SupplierName в DataGridView
        if (!dataGridViewFull.Columns.Contains("SupplierName"))
        {
            DataGridViewTextBoxColumn supplierNameColumn = new
DataGridViewTextBoxColumn
            {
                Name = "SupplierName",
                HeaderText = "Имя Поставщика",
                ReadOnly = true
            };
            dataGridViewFull.Columns.Add(supplierNameColumn);
        }

        dataGridViewFull.Columns["SupplierName"].DisplayIndex = 1;
        dataGridViewFull.Columns["PurchaseID"].Visible = false;
        dataGridViewFull.Columns["SupplierID"].Visible = false;
        dataGridViewFull.Columns["SupplierName"].Visible =
checkBoxAllSuppliers.Checked;

        // Настройка заголовков столбцов
        dataGridViewFull.Columns["PurchaseDate"].HeaderText = "Дата
Закупки";
        dataGridViewFull.Columns["PurchaseReason"].HeaderText = "Причина
Закупки";
        dataGridViewFull.Columns["TotalQuantity"].HeaderText = "Общее
Количество";
        dataGridViewFull.Columns["TotalPrice"].HeaderText = "Общая
Стоимость";

        dataGridViewFull.Columns["PurchaseDate"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;
        dataGridViewFull.Columns["PurchaseReason"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
        dataGridViewFull.Columns["TotalQuantity"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;
        dataGridViewFull.Columns["TotalPrice"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;

        int grandTotalQuantity = 0;
        decimal grandTotalPrice = 0;

        foreach (DataRowView purchaseRow in view)
        {
            int purchaseId = (int)purchaseRow["PurchaseID"];

```

```

        var details =
computerCompanyDBDataSet.PurchaseDetails.AsEnumerable()
            .Where(d => d.Field<int>("PurchaseID") == purchaseId);

        int totalQuantity = details.Sum(d => d.Field<int>("Quantity"));
        decimal totalPrice = details.Sum(d => d.Field<int>("Quantity")
* d.Field<decimal>("UnitPrice"));

        purchaseRow["TotalQuantity"] = totalQuantity;
        purchaseRow["TotalPrice"] = totalPrice;

        grandTotalQuantity += totalQuantity;
        grandTotalPrice += totalPrice;
    }
    labelTotals.Text = $"Итого: Количество = {grandTotalQuantity},
Сумма = {grandTotalPrice}";

// Проверка на наличие хотя бы одной строки перед добавлением
итоговой строки
if (view.Count > 0)
{
    // Добавление итоговой строки
    DataRowView totalRow = view.AddNew();
    totalRow["PurchaseReason"] = "Итого";
    totalRow["TotalQuantity"] = grandTotalQuantity;
    totalRow["TotalPrice"] = grandTotalPrice;
}

dataGridViewFull.Refresh();
listBoxItems.Items.Clear();
}
catch (Exception ex)
{
    MessageBox.Show("Произошла ошибка: " + ex.Message);
}
}

private void MainForm_Load(object sender, EventArgs e)
{
    LoadData();
}

e) private void checkBoxAllSuppliers_CheckedChanged(object sender, EventArgs
{
    comboBoxSupplier.Enabled = !checkBoxAllSuppliers.Checked;
}

private void checkBoxAllTime_CheckedChanged(object sender, EventArgs e)
{
    dateTimePicker1.Enabled = !checkBoxAllTime.Checked;
    dateTimePicker2.Enabled = !checkBoxAllTime.Checked;
}

private void ShowForm(bool reportFlag)
{
    int? supplierId = null;
    string supplierName = null;
    DateTime? startDate = null;
    DateTime? endDate = null;

    if (!checkBoxAllSuppliers.Checked)
    {
        if (comboBoxSupplier.SelectedItem != null)

```

```

        {
            supplierId =
(int)((DataRowView)comboBoxSupplier.SelectedItem)["SupplierID"];
            supplierName =
((DataRowView)comboBoxSupplier.SelectedItem)["SupplierName"].ToString();
        }
    }
    if (!checkBoxAllTime.Checked)
    {
        startDate = dateTimePicker1.Value;
        endDate = dateTimePicker2.Value;
    }

    if (reportForm == null || reportForm.IsDisposed)
    {
        reportForm = new ReportForm(startDate, endDate, supplierId,
supplierName, reportFlag);
        reportForm.FormClosed += (s, args) => reportForm = null;
        reportForm.ShowDialog(this);
    }
}

private void buttonReport_Click(object sender, EventArgs e)
{
    ShowForm(true);
}

private void buttonFullReport_Click(object sender, EventArgs e)
{
    ShowForm(false);
}

private void toolStripButtonPurchases_Click(object sender, EventArgs e)
{
    purchasesForm = new PurchasesForm();
    purchasesForm.ShowDialog();
}

e) private void toolStripButtonPurchaseDetails_Click(object sender, EventArgs
{
    purchaseDetailsForm = new PurchaseDetailsForm();
    purchaseDetailsForm.ShowDialog();
}

private void toolStripButtonSuppliers_Click(object sender, EventArgs e)
{
    suppliersForm = new SuppliersForm(true);
    suppliersForm.ShowDialog(this);
}

private void toolStripButtonComponents_Click(object sender, EventArgs e)
{
    componentsForm = new ComponentsForm();
    componentsForm.ShowDialog();
}

private void toolStripButtonCategories_Click(object sender, EventArgs e)
{
    categoriesForm = new CategoriesForm();
    categoriesForm.ShowDialog();
}
}
}

```